Coarse-to-Fine Text Generation on a Laptop: A Native MLX Study of Next-Concept Modeling

Jan Music
Independent

Abstract

Next-Concept Modeling (NCM) generates a sequence through successive levels of discrete latent codes, predicting each level in parallel. The method is by Kumbong et al.; we implement it independently in MLX and study synthetic language, Sudoku and short synthetic stories on Apple Silicon and NVIDIA GPUs. On a finite synthetic grammar, a 5M-parameter generator achieves lower small-batch latency than a similarly sized autoregressive generator, with lower sentence validity (89.8% versus 98.6%). Two Sudoku training seeds use the same 6.44M-parameter generator recipe and 16.384M presentations each. Retained greedy evaluation aggregates report easy/medium/hard solve rates of 98.10/83.40/40.30% for seed 0 and 97.25/79.60/33.25% for seed 1, with 2,000 boards per difficulty; two seeds do not establish population training variability or its cause. Original NCM reports 99.50/96.50/86.10% with a 25.46M generator, 51.2M presentations and a different task setup, so the comparison establishes neither task parity nor matched-cost efficiency. Missing per-board predictions and remote provenance manifests limit independent audit of the Sudoku runs. This version adds two bounded results. First, an inference-only MLX port of the released TinyStories and Sudoku checkpoints agrees with upstream PyTorch on CPU in the tested computations: the largest compared fp32 TinyStories logit difference is 0.100 at logit magnitudes up to 853, and 3.2e-10 in a composed fp64 diagnostic. The converted Sudoku checkpoints solve 99.40/96.30/86.20% of 2,000 boards per difficulty. This concerns upstream-trained weights, not their training. Second, we train small TinyStories models (an 11.8M-value generator on a frozen 15.5M-value codec) and ablate conditioning corruption with two generator seeds that share one codec. The codec reconstructs held-out windows accurately (99.816% of non-padding tokens). Clean teacher-forced code cross-entropy and free-running quality rank the three corruption schedules in opposite order in both seeds. Under a fixed external scorer (GPT-2 large), generative perplexity for strong, reduced and no corruption is 47.897/54.923/97.776 for generator seed 0 and 45.568/55.114/93.833 for seed 1. An MLX autoregressive baseline with the same block stack scores 14.676 and 14.311, the released TinyStories checkpoint 16.259 and held-out text 8.529. Total training compute is not matched, the corpus is synthetic, generation is unconditional and no conditional task is tested. We characterize the work as a partial implementation, a small-model Sudoku success and a documented failure of free-running text quality at pilot scale. Version 0.13 adds a scaling study in the same TinyStories setup, with one frozen codec and one external scorer. With the generator raised from 11.8M to 49.6M and 87.4M parameters at the original 960,000 windows, the ratio of NCM to AR generative perplexity falls from 3.33 to 2.43 and 2.38; with four times the data (and bf16 training) it is 2.21 and 2.13 for the two larger sizes and 3.05 and 2.89 for two seeds of the 11.8M generator. Most of the change comes from the first size step and from the extra data; the step from 49.6M to 87.4M is smaller than the difference between two training seeds of the 11.8M generator. The best NCM scores 24.404 against 11.484 for its AR

baseline. Optional bf16 mixed-precision training matches the fp32 quality of the 11.8M recipe closely (clean code cross-entropy 2.24705 against 2.24708–2.24909; generative perplexity 49.091 against 45.568–48.964 for three fp32 runs, 0.13 above the highest). Several follow-ups give no gain: a matched-constant corruption control (47.052), stronger corruption (55.329 and 53.297 for two seeds), equal level loss (47.543 and 45.740) and a sampling-policy study that attributes roughly 5–10% of the gap (in log terms) to sampling. A small FineWeb-Edu run is reported only as a pipeline check on web text, not as a result. The scaling rows are single-seed except one, mix fp32 and bf16, compare unequal compute and use synthetic prose, so they describe a trend under stated conditions and are not a scaling law. An inference-only port of the released XSum generator matches upstream PyTorch in CPU parity checks with greedy decoding and scores ROUGE-1/2/L of 34.93/12.71/27.94 on the 11,334-article test set, against 35.0/12.8/28.0 quoted from Kumbong et al.

Keywords: coarse-to-fine generation, residual vector quantisation, parallel decoding, Apple Silicon, MLX

Status. Unreviewed working paper, version 0.13 (version DOI: 10.5281/zenodo.23953213; the previous version, 0.12, is 10.5281/zenodo.23897923). Every number here is copied from a run in the code repository (`runs/`, summarised in `paper/data/results.json` and `docs/research/tinystories-scaling.json`), from the retained evidence extracts under `docs/research/`, or quoted from the cited paper and marked as quoted. The TinyStories evidence of Section 10 is retained as described there; its sample-level records are not yet part of a clean public checkout, and the evidence for Section 11 is attached to the paper release as an archive (see Code and reproducibility). Experiments still running are marked *in progress* and never estimated. Appendix C lists the changes between versions.

1 Reading guide

This paper has two audiences. **If you are new to language models**, read Section 2 (the idea in plain language) and Section 3 (glossary), then the boxed summary at the start of each later section. **If you build models**, Sections 5–7 give the method with every formula explained, the Apple Silicon implementation and the experiments. Section 8 tells the story of the project run by run: what each experiment (E0, E1, …) asked, what changed and what we learned. Section 9 reports how the released checkpoints of the original authors run in our MLX port, and Section 10 reports our own TinyStories experiments, including the corruption ablation and its seed repeat. Section 11 reports the follow-ups: scaling the generator and the data, bf16 parity, sampling and corruption variants that did not help, and a web-text pilot.

2 The idea in plain language

2.1 How a chat model writes

A chat model such as ChatGPT or Qwen writes one *token* (a word or a piece of a word) at a time. To write the 100th token it runs the whole model once, reads what it has written so far and picks the next token. A 100-token answer therefore needs 100 full runs of the model, one after the other. At small batch sizes, autoregressive decoding often spends much of its time moving model weights from memory. The balance depends on the model, hardware, cache and implementation; these experiments measure latency, not memory-bandwidth utilisation.

2.2 How next-concept modeling writes

NCM (Kumbong et al. 2026) writes differently. It first produces a single, very rough “concept” for the whole text, then refines it in a few rounds. Each round adds roughly twice as much detail, and **every piece of detail in a round is produced at the same time**. Figure 1 shows a real example from this project: a 32-character sentence written in six rounds (levels). After level 0

the rough shape is there (“eue likes one oed egg.”); by level 2 the words are settled; later levels only change single letters.

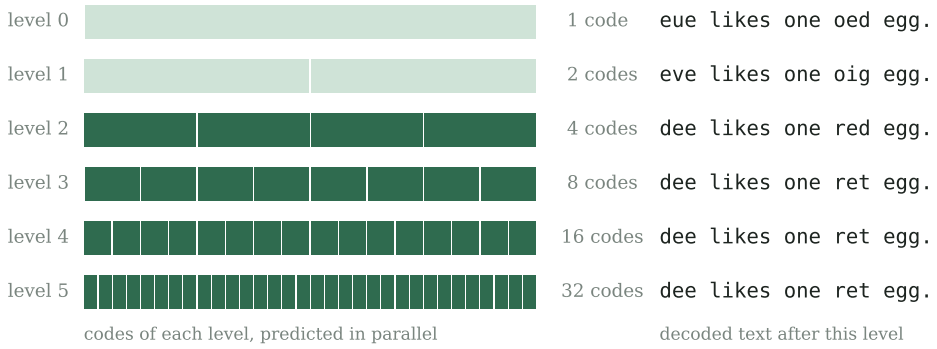


Figure 1: A real generation from experiment E1. Left: the codes of each level; all codes in one row come from one model run. Right: the text obtained by decoding everything up to that level. Six model runs write the sentence; a word-by-word model needs 32. The sample also shows a failure: level 3 turns “red” into “ret”, and nothing later repairs it.

The number of rounds is fixed by the model configuration (six in TinyWorld, twelve in the original paper’s Sudoku model), so NCM needs far fewer model runs than a word-by-word model. That is where its speed comes from.

2.3 Two parts: a codec and a generator

NCM has two learned parts. An analogy from image files helps.

- The **codec** is like an image format with progressive loading: a JPEG that first shows a blurry picture, then sharpens. It learns to turn a sentence into a stack of codes, from one coarse code to one code per letter, and to turn the codes back into the sentence. It is trained first, on its own, with reconstruction losses. Full-hierarchy reconstruction is measured separately from generation quality.
- The **generator** is the creative part. It learns to *invent* new code stacks one level at a time: level 0, then level 1 given level 0, and so on. The codec then turns the finished stack into text.

2.4 Why it matters, and the catch

Fewer, wider model runs suit a laptop GPU: one run reads the weights once and does a lot of arithmetic with them. The catch is that all codes of a level are chosen at the same moment, so neighbouring pieces cannot see each other’s choice. If two pieces of a level disagree, the result can be wrong in a way a word-by-word model would avoid. Much of this paper measures when that matters.

2.5 What we did

We rebuilt NCM from scratch for Apple’s MLX framework (Hannun et al. 2023), tested it on a small invented language and on Sudoku puzzles (the original paper’s showcase), and compared it with a character-level AR model of similar generator size and with a large chat model. The speed advantage holds on a laptop. On Sudoku, two training seeds solve 97.25–98.10% of easy boards but only 33.25–40.30% of hard boards. Original NCM reports 99.5% and 86.1%, respectively, under a different setup. A historical comparison matches generator presentations against our AR baseline; Section 8 records the experiments.

Two later additions concern text. We wrote an inference-only port that runs the original authors’ released checkpoints and checked it against their PyTorch code (Section 9). We also trained small models of our own on TinyStories, a corpus of short synthetic children’s stories (Section 10). Those models reconstruct held-out story text accurately but generate stories that an external language model scores far worse than a word-by-word baseline built from the same transformer blocks. A further set of follow-ups (Section 11) asks whether the gap to the autoregressive

baseline shrinks with a larger generator and more data. With the codec held fixed it does, from a ratio of about 3.3 to about 2.1, but it does not close, and the evidence is single-seed and limited. We also added the released XSum generator to the port (Section 9). NCM is a generation method used here for three tasks, not a Sudoku-specific expansion of the term or an instruction-following language model.

3 Glossary

Table 1: Terms used in this paper.

Term	Meaning
Token	The unit a model reads and writes. Here: one character (TinyWorld), one Sudoku cell or one byte-pair piece of text (TinyStories).
Autoregressive (AR) model	A model that writes one token at a time, each conditioned on all previous ones. Chat models are AR models.
Forward pass, NFE	One run of a model on its input. NFE (number of function evaluations) counts forward passes per generated sample.
Parameters	The learned numbers inside a model; "6.4M parameters" means 6.4 million.
Embedding, latent vector	A list of numbers (128–512 here) that represents a token or a position inside the model.
Vector quantisation (VQ)	Replacing a vector by the nearest entry of a fixed list, the *codebook*, so it can be stored as one integer, the *code*.
Codebook, code	The list of allowed vectors at one level (16 or 128 entries here) and the index of one entry.
Level	One round of refinement. Level k has l_k codes; level 0 is the coarsest.
Residual	What is left over: the difference between the true vector and the approximation built so far.
Codec (tokenizer, VQ-VAE)	Stage 1 of NCM: encoder, quantiser and decoder that turn text into codes and back.
Generator	Stage 2 of NCM: the transformer that predicts the codes of each level.
Transformer, attention, mask	The standard neural network for sequences. *Attention* lets each position read information from other positions; a *mask* says which positions may be read.
Teacher forcing	Training on the *true* previous outputs instead of the model's own. Fast, but the model never practises recovering from its own mistakes.
Conditioning corruption	Replacing some true inputs with wrong ones during training so the model learns to cope with its own mistakes.
Cross-entropy (CE), nats	The training loss: how surprised the model is by the right answer, in natural-log units. Lower is better; $\ln 16 \approx 2.77$ nats is a uniform guess among 16 codes.
Perplexity	e^{CE}, the effective number of choices. Code perplexity 1 means a level always uses the same code and carries no information.

Term	Meaning
BPE	Byte-pair encoding: a fixed vocabulary of frequent word pieces (8,192 here) into which text is split.
Clean code CE	Generator cross-entropy per code when every coarser level is supplied correctly (teacher forcing), on held-out windows. It is not a measure of free-running generation.
Generative perplexity	Perplexity that an *external* language model (GPT-2 large) assigns to text a model generated on its own. Lower is better; it is not the generating model's own likelihood.
Temperature, greedy	Sampling randomness. Temperature 0 (greedy) always takes the most likely choice.
Batch, latency, p50	A batch is several samples generated together; latency is the time for one batch; p50 is the median over repeated runs.
KV cache	A store of past attention values that speeds up AR generation.
EMA	Exponential moving average: a running average that slowly forgets old values. Used to update codebooks.
Straight-through estimator	A trick to train through a step that has no gradient (like picking the nearest code) by treating it, for the gradient only, as the identity.
MLX, Metal	Apple's machine-learning array framework, and the GPU interface it runs on.
fp32, bf16	32-bit and 16-bit (bfloat16) floating-point formats.
E0, E1, …	Experiment numbers, in the order the experiments were run (Section 8).

4 Background

Residual vector quantisation. A vector quantiser maps a continuous vector to the nearest of V codebook entries (Oord et al. 2017). Residual quantisation applies several quantisers in sequence, each to what the previous ones left over (Lee et al. 2022; Zeghidour et al. 2021); it underlies modern neural audio codecs.

Next-scale prediction. VAR (Tian et al. 2024) combines residual quantisation with resolution: the first quantiser sees a 1×1 summary of an image, later ones 2×2, 3×3, up to full resolution, and a transformer generates one scale per step. NCM applies the idea to 1-D token sequences (Kumbong et al. 2026).

Diffusion language models refine positions of a sequence over T denoising steps (Sahoo et al. 2024). Full-sequence samplers process length L at each step; NCM's early levels process fewer code positions, although prefix and decoder work also contribute. The NCM paper reports quality at or above strong diffusion language models with 10–31× faster generation under its H100 benchmark protocol (Kumbong et al. 2026).

5 Method

> **In short.** The codec learns to describe a sequence with a stack of codes, coarse to fine. The generator learns to predict each level's codes from the levels above it. Generation runs the generator once per level, then decodes.

We follow the NCM design (Kumbong et al. 2026) and its released Sudoku configuration; Table 2 lists where we differ. A running toy example uses a sequence of $L = 8$ tokens, levels $l = (1, 2, 4, 8)$ and vectors of width $C = 2$.

5.1 Codec: from tokens to a stack of codes

Encoder. Each token x_i and its position i are looked up in two tables, added and normalised:

$$z_i = \mathrm{Norm}\big(W_{\mathrm{tok}}[x_i] + W_{\mathrm{pos}}[i]\big), \qquad z \in \mathbb{R}^{L \times C}.$$

Reading it: W_{tok} has one learned row per possible token and W_{pos} one per position; z is the whole sequence as L vectors of C numbers. "Norm" rescales each vector (LayerNorm), or groups of channels across the sequence (GroupNorm, as upstream), without a learned scale, because the quantiser compares squared distances and a learned scale could grow without bound. In the toy example z is an 8×2 table of numbers.

Quantisation, level by level. The quantiser keeps a *residual* r, initially all of z, and for each level $k = 0, \ldots, K-1$ does four things:

$$\underbrace{x_k = D_k\, r_k}_{\text{1. shrink}}, \qquad \underbrace{c_k = \arg\min_j \|x_k - E_k[j]\|^2}_{\text{2. pick nearest codes}},$$

$$\underbrace{q_k = \phi_k\big(U_k\, E_k[c_k]\big)}_{\text{3. expand and refine}}, \qquad \underbrace{r_{k+1} = r_k - \mathrm{sg}(q_k)}_{\text{4. subtract}}.$$

Reading it:

1. **Shrink.** D_k averages the residual down to l_k positions (area pooling). Toy example: for level 1 ($l_1 = 2$), positions 1–4 are averaged into one vector and 5–8 into another.
2. **Pick.** For each of the l_k shrunk vectors, find the nearest of the V codebook entries $E_k[1..V]$; its index is the code. c_k is a list of l_k integers, for example $(5, 12)$.
3. **Expand and refine.** Look the codes back up, stretch them to L positions by linear interpolation (U_k), and apply $\phi_k(h) = (1-\rho)\, h + \rho\, \mathrm{conv}_k(h)$: a small learned convolution blended half and half ($\rho = 0.5$) with its input, which smooths the blocky stretched result.
4. **Subtract.** Remove what this level explained, so the next level only describes what is left. sg (stop-gradient) means the subtraction is not differentiated.

The reconstruction after level k is the sum of everything explained so far:

$$\hat{z}_k = \sum_{j \le k} q_j, \qquad \hat{z} = \hat{z}_{K-1} \approx z.$$

Decoding $\hat{z}_k$ for $k < K-1$ gives the "after level k" texts of Figure 1.

Codebook updates. Codebooks are not trained by gradients. After each step every entry moves towards the average of the vectors assigned to it:

$$N_j \leftarrow \gamma N_j + (1-\gamma)\, n_j, \qquad m_j \leftarrow \gamma m_j + (1-\gamma) \sum_{i:\, c_i = j} x_i, \qquad E[j] = m_j / \tilde{N}_j.$$

Reading it: n_j counts how many vectors picked entry j in this batch, N_j is its running average with decay $\gamma = 0.99$, m_j the running sum of those vectors, and $\tilde{N}_j$ is N_j with a tiny amount added to every count (Laplace smoothing) so nothing is divided by zero. Entries nobody uses (N_j below a threshold) restart at a random input vector.

Losses. The decoder, one transformer block, reads $\tilde{z} = z + \mathrm{sg}(\hat{z} - z)$. Numerically this *is* $\hat{z}$, but its gradient flows into z as if quantisation were the identity (the straight-through estimator), so the encoder learns. The training loss is

$$\mathcal{L}_{\mathrm{codec}} = \underbrace{\mathrm{CE}\big(\mathrm{Dec}(\tilde{z}), x\big)}_{\text{rebuild the tokens}} + \underbrace{\frac{1}{K} \sum_k \Big(\beta\, \|\mathrm{sg}(\hat{z}_k) - z\|^2 + \|\hat{z}_k - \mathrm{sg}(z)\|^2 \Big)}_{\text{VQ loss}}, \quad \beta = 0.25.$$

Reading it: the first term asks the decoder to rebuild the exact tokens. The *commitment* term (β) pulls the encoder output towards what the codes can express; the *fit* term trains the refinement convolutions ϕ_k so each partial sum $\hat{z}_k$ is as close to z as possible.

Fine-level dropout. Half of the time the decoder receives $\hat{z}_k$ for a random $k < K-1$ instead of the full $\hat{z}$. It must then make sense of a coarse-only description, which is why every prefix of levels decodes to a plausible sequence: a form of nested dropout (Rippel et al. 2014).

Learned level 0. Averaging an entire Sudoku grid into one vector gives nearly the same vector for every grid, so level 0 carries no information (Section 7.2). Upstream replaces D_0 and U_0 by a strided convolution and a transposed convolution initialised to behave like pooling and repetition. Picking a code has no gradient, so the gradient reaches them through a straight-through estimator: q_0 is built from $x_0 + \mathrm{sg}(E_0[c_0] - x_0)$, whose value is the code vector and whose gradient is that of x_0.

5.2 Generator: predicting one level from the levels above

The generator is a transformer that reads one long row of vectors:

$$[\ \text{prefix}\ (P)\ |\ \text{level-0 queries}\ (l_0)\ |\ \text{input}_1\ (l_1)\ |\ \ldots\ |\ \text{input}_{K-1}\ (l_{K-1})\ ], \qquad \text{input}_k = D_k\,\hat{z}_{k-1}.$$

Reading it: the *prefix* holds the conditioning, for Sudoku the 81 cells of the puzzle (blanks as 0). Level 0 has nothing above it, so it reads learned query vectors. Every other level reads the reconstruction from all coarser levels, shrunk to its own length. Toy example: input_2 is $\hat{z}_1$ (levels 0 and 1 combined) averaged down to 4 positions. Head k turns each of the l_k positions of its block into a probability for each of the V codes of level k.

Training loss. With the true codes c_k of a training example,

$$\mathcal{L}_{\text{gen}} = \frac{\sum_k l_k \cdot \mathrm{CE}_k}{\sum_k l_k}, \qquad \mathrm{CE}_k = -\frac{1}{l_k}\sum_{i=1}^{l_k} \log p_\theta\left(\, c_{k,i} \mid \text{input}_k, \text{prefix}\,\right).$$

Reading it: CE_k is the average surprise at the true codes of level k; weighting by l_k makes every code count equally (the paper's $\alpha = 0$ setting (Kumbong et al. 2026, App. F.2.3)). A uniform guess among 16 codes scores $\ln 16 \approx 2.77$.

Attention pattern. Positions of level k may read the prefix and the other positions of level k, nothing else (*block-diagonal* attention); coarser levels reach level k only through input_k. Two consequences follow. Training on the whole row at once computes exactly what level-by-level generation computes (tested, Section 6.2). And each level's codes are chosen without seeing each other's choices, the source of both the speed and errors like "red" → "ret". RoPE (Su et al. 2021) positions restart at zero in every level, a learned level embedding tells levels apart, and queries and keys are RMS-normalised.

Conditioning corruption. In training the generator reads true coarser codes; in generation it reads its own, sometimes wrong, codes. Upstream narrows this gap by replacing a fraction p_k of the conditioning codes of level k with random codes of the same level, keeping the true codes as targets, with p_k falling linearly across levels (Kumbong et al. 2026, Sec. 3.4):

$$p_k = p_{\text{first}} + (p_{\text{last}} - p_{\text{first}})\,\frac{k}{K-2}, \qquad k = 0, \ldots, K-2.$$

This is the coarse-to-fine analogue of scheduled sampling (Bengio et al. 2015). The paper notes that corruption "based on the model's actual errors may better match inference conditions" but does not test it; experiment E9 tests it and finds no gain at our scale (Section 8). Section 10 varies the corruption schedule itself on TinyStories text.

5.3 Generation

For $k = 0, \ldots, K-1$: run the generator on input_k and the prefix, sample all l_k codes of level k at once, add their contribution q_k to the reconstruction and shrink it into input_{k+1}. Finally decode $\hat{z}_{K-1}$ to tokens. Cost: K generator passes and one decoder pass, independent of how long each level is.

Table 2: Initial E6 settings versus released Original NCM Sudoku settings (Kumbong et al. 2026, Tables 5–6). Later runs change exposure and corruption; see Appendix A.

Component	Original NCM (Kumbong et al.)	MLX starting recipe (E6)
Tokens per board	89 (cells and separators)	81 (cells)
Levels × codes	12×16, $l_{K-1} = 89$	12×16, $l_{K-1} = 81$
Level-0 sampler	learned (one layer, as 89 is prime)	learned from E6, one layer
Codec training	10,000 updates × batch 1,024, lr 5e-4	3,000 × 128, lr 2e-4
Generator	8×512 (25.5M), 50,000 updates × 1,024, lr 3e-4	8×256 (6.4M), 4,000 × 128, lr 6e-4
Conditioning corruption	yes, linear schedule	from E8 ($0.30 \rightarrow 0.05$)
Models	one per difficulty	one for all difficulties

6 Implementation on Apple Silicon

> **In short.** Pure MLX and NumPy, no code copied from upstream. Two findings matter beyond this project: Apple GPUs compute fp32 matrix products at reduced precision, and block-diagonal attention is faster computed per level than through a mask.

6.1 Framework mapping

Codebooks and their running averages are frozen MLX arrays updated outside automatic differentiation; the shrink and stretch operators D_k, U_k are precomputed matrices; attention uses MLX's fused `scaled_dot_product_attention`. Weights are stored as safetensors. The correspondence to upstream is documented component by component in `docs/research/upstream-map.md`.

6.2 Exactness and Metal precision

A test asserts that level-by-level generation produces the same logits as the training forward. On the Metal GPU the two disagreed by about 5×10^{-4}. Against a float64 NumPy reference, the same fp32 computation had an error of 1.4×10^{-2} on the GPU and 2×10^{-6} on the CPU: fp32 matrix products on Metal run at reduced internal precision, about 10^{-3} relative. Exactness tests therefore run on the CPU, and a separate test checks the GPU path within 5×10^{-3}. This matters for anyone writing equivalence tests for MLX models; sampling quality is evaluated separately, and these exactness checks do not establish the quality effect of precision.

6.3 Segmented block-diagonal attention

With a mask, attention over the full Sudoku row forms a 395×395 score matrix per head, most of it masked out. Computing attention separately for the prefix and for each level (keys and values: prefix plus that level) gives the same result with $P^2 + \sum_k l_k(P + l_k) \approx 48,000$ scores instead of 156,000. In training (width 256, batch 128, compiled) step time fell from 1.59 s to 1.18 s and peak memory from 18.7 GB to 16.4 GB, measured back to back. A test checks equality with the masked computation.

6.4 Compilation and precision

Compiling the training step with `mx.compile` cut step time by about 1.3× (1.22 s → 0.92 s at width 256, batch 128). Casting the generator to bfloat16 gave a further 1.4–1.8× in a first test but saved no memory, because the frozen fp32 codec promotes the generator's inputs back to fp32; mixed precision is not used in the experiments reported here.

6.5 Interrupted training and reproducibility

Long runs need recoverable training state as well as model weights. Codec and generator snapshots now store optimizer moments and step, random state, shuffled-batch position, elapsed training time and model parameters; codec snapshots include VQ running averages and generator snapshots retain the best-validation model. Files are replaced atomically, with a preceding snapshot retained. A pause signal saves after the current step; a hard crash recovers from the last completed snapshot. Short CPU tests give identical saved state for uninterrupted and resumed training, including the compiled generator path. This does not establish equality across hardware or backends. Rented-pod recovery also depends on copying snapshots outside the container before it disappears.

E17 exposed the distinction: closing its terminal killed training after the last saved step, 30,000 of 32,000. That legacy checkpoint contained weights alone. Its continuation retains the learning-rate and batch positions but resets optimizer moments and random state, so it must be reported as a weights-only recovery. The recovered model solves 71.25 / 31.75 / 3.70% of easy/medium/hard boards, close to E13, after 2,000 continuation steps taking 1,247.53 s. This is consistent with puzzle construction affecting transfer, but the reset prevents a controlled causal conclusion. The operational procedure is documented in `docs/RECOVERY.md`.

7 Experiments

> **In short.** On a small invented language NCM is faster and somewhat less accurate than an AR model with similar generator size. On Sudoku our AR result is near the Original AR baseline; our NCM does not match Original NCM's reported result.

Runs use an Apple M5 Pro (64 GB unified memory, macOS 26.5), an RTX 3080 or a rented A40, as labelled for each experiment, with MLX 0.32.3. Other GPU workloads often ran on the Mac; timings this affects are marked. The TinyStories experiments of Section 10 and the parity checks of Section 9 ran on the Mac.

7.1 TinyWorld: an invented language

TinyWorld has 15,360 sentences of the form *name verb count adjective noun*, with number agreement ("one red cat", "two red cats"), at most 32 characters. 12,000 sentences train the models; the rest are held out. A sentence is *valid* if it parses and agrees. The codec (6 levels, $l = 1, 2, 4, 8, 16, 32, 128$ codes each, 1.3M parameters) rebuilds 100% of held-out sentences exactly. The NCM generator (5.0M parameters) is compared with a character-level AR transformer of the same width and depth (4.8M parameters) with a KV cache.

Table 3: TinyWorld, NCM and AR with similar generator size (E1); NCM also retains its codec.

	MLX implementation	MLX AR baseline
Valid, temperature 1.0	89.8%	98.6%
Valid, temperature 0.7	95.8%	99.8%
Valid and not in the training set, temperature 1.0	16.3%	23.5%
Forward passes per sequence	6 + 1 decode	32
Median latency, batch 1 / 16 / 256	4.5 / 5.8 / 37.8 ms	30.5 / 22.9 / 49.8 ms

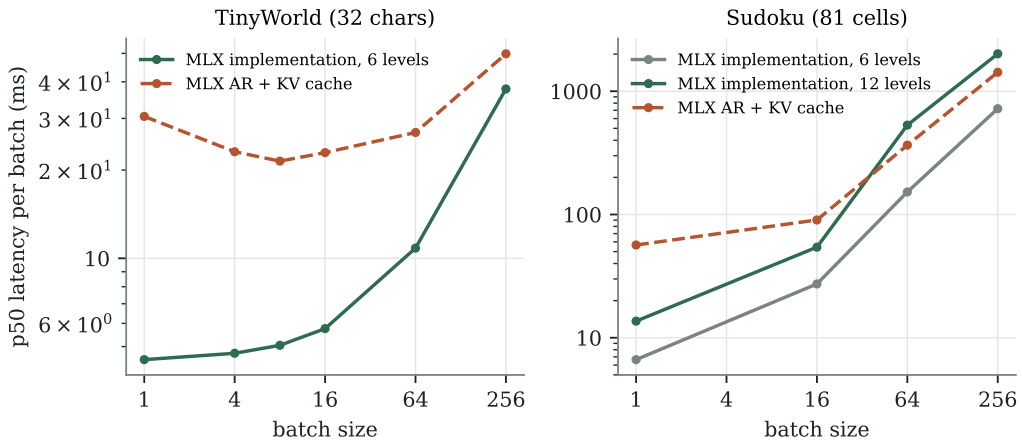


Figure 2: Median latency per batch for NCM and AR models. Left: TinyWorld. Right: Sudoku. Both axes logarithmic.

NCM is less accurate and faster. Its errors are local spelling slips; agreement, a sentence-wide rule, holds in 99.8% of well-formed samples. Forcing the true codes for the first k levels and sampling the rest (E2) shows that errors arise at every level, not only the finest. Letting level k also read coarser levels (block-causal attention, E3) raised validity at temperature 1 from 89.8% to 92.2% (n = 1,024, standard error about 0.9 points) at 1.5–1.9× the latency.

7.2 Sudoku

Comparison labels. "MLX implementation" means the independently trained NCM model in this repository. "Original NCM" means Kumbong et al.'s published model and checkpoint results (Kumbong et al. 2026). The "MLX AR baseline" is trained here; the "Original AR baseline" is quoted from their study. The underlying method is NCM in both implementations.

Each board is 81 digits; the puzzle (blanks as 0) is the prefix and the solution is the target. Training uses 200,000 synthetic puzzles from randomised backtracking with 28–45 givens. Test sets are the 2,000 easy, medium and hard boards (40, 35 and 30 givens) published with the NCM paper. A board counts as *solved* if the output is a valid grid that keeps every given. Decoding is greedy.

Table 4: Sudoku, share of boards solved. Params counts generators only for NCM; the codec is additional. Paper rows are reference results from (Kumbong et al. 2026, Table 3). Budgets differ except for the explicitly matched E11 comparison.

Model	Params	Easy	Medium	Hard
MLX implementation, E4–E6	6.4M	0.0%	0.0%	0.0%
MLX implementation, E8 (+ corruption)	6.4M	6.2%	0.35%	0.0%
MLX implementation, E11 (+ 4× data)	6.4M	55.2%	18.1%	1.6%
MLX implementation, E13 (+ stronger corruption)	6.4M	70.9%	31.2%	4.0%

Model	Params	Easy	Medium	Hard
MLX implementation, E14 (+ 16× data)	6.4M	87.4%	53.4%	11.2%
MLX implementation, E15	6.4M	96.25%	76.65%	31.90%
MLX implementation, E18 (base data)	6.4M	84.25%	49.10%	10.55%
MLX implementation, E19 pilot	6.4M	90.20%	59.75%	16.55%
MLX implementation, E19 funded (seed 0)	6.4M	98.10%	83.40%	40.30%
MLX implementation, E20 (seed 1)	6.4M	97.25%	79.60%	33.25%
MLX AR baseline	6.3M	13.8%	3.0%	0.4%
MLX AR baseline, E11′s budget (2M sequences)	6.3M	20.5%	4.9%	0.5%
Qwen3.8 27B, 4-bit, no reasoning (50 boards each)	27B	0%	0%	0%
Original AR baseline (Kumbong et al.)	25M	14.6%	5.1%	1.0%
MDLM (reported by Kumbong et al.)	29M	92.0%	77.1%	30.2%
Original NCM (Kumbong et al.)	25M	99.5%	96.5%	86.1%
Released Sudoku checkpoints (Kumbong et al.), run in the MLX port on the M5 Pro	25M	99.40%	96.30%	86.20%

The first AR row is within a few percentage points of the Original AR baseline, but this numerical proximity does not establish a reproduction of its model or training recipe. Without conditioning corruption (E4–E6) our NCM solves nothing; with it (E8) it solves 6.2% of easy boards and fills 74.7% of blank cells correctly, more than the AR model's 71.0%, while still solving fewer boards. E11 solves 55.2% of easy boards; at the same 2.048M generator presentations, the AR model solves 20.5%, 4.9% and 0.5%. This matches generator data exposure, not total training cost, FLOPs or system size. Section 8 records the changes. The last row evaluates the original authors' released, upstream-trained weights through our inference port (Section 9); it is separate from the E19 and E20 models trained here.

The shared E6 codec contains 13,816,704 gradient-trained values, 49,152 EMA-trained codebook values and 49,344 EMA accumulators: 13,915,200 saved scalar values in total. Its learned first-level down/up convolutions account for 10,617,344 trainable values. The 6,440,128 generator values are additional; generator-size comparisons exclude the codec. Older rows preserve

historical rounding; E15 and the newly completed runs retain their two-decimal solved rates. Each test set has 2,000 boards. Most recipes use one training seed; E19-funded and E20 provide two seeds of the same scientific recipe. Repeated sampling seeds do not quantify training variability.

Historical synchronized timings use temperature 1.0 while Sudoku quality uses greedy temperature 0.0. Both samplers remain uncompiled; `mx.compile` is used only for generator training when configured. Latency therefore does not directly time the solved-board decoding setting. The benchmark includes the output host copy, prefix work and final decoder, and records shared-device contention; cross-hardware timing ratios are not controlled comparisons.

Smaller-generator result. Easy-board accuracy is within 1.40 percentage points of Original NCM (98.10% versus 99.50%) with a 6.44M versus 25.46M generator: 3.95× smaller and 3.125× fewer generator presentations. This is a useful small-model result, not overall task parity: medium and hard remain 13.10 and 45.80 points behind. The separate codec is excluded from this size ratio, and the recipes and representation differ. E20 retains the same generator size and exposure but its easy score is 2.25 points below Original NCM; its medium/hard gaps are 16.90/52.85 points. The best-seed proximity on easy boards is not a reproducibility guarantee.

7.2.1 Completed E18 and E19 experiments

E18 tests stronger corruption on the base 200,000-puzzle corpus at E15′s 8.192M-presentation budget. Its 84.25 / 49.10 / 10.55% solve rates remain below both E14 and E15. Stronger corruption is therefore not a universal improvement, and this negative result remains part of the evidence.

The short E19 pilot uses the frozen E6 codec and 8M cached unique puzzles in addition to the base corpus. It completes 23,895 updates at batch 128 (3,058,560 presentations), solving 90.20 / 59.75 / 16.55%. E19-funded starts a fresh generator with the same corpus and a separate 128,000-update cosine schedule. At 16,384,000 presentations it solves 98.10 / 83.40 / 40.30%, improving on E15 by 1.85 / 6.75 / 8.40 percentage points. The exposure doubles while batch size, corpus and schedule differ. These comparisons establish outcomes of whole recipes, not a controlled effect of any one change.

Table 5: Completed training and checkpoint selection (training-loop seconds).

Run	Updates × batch	Presentations	Best step	Train seconds
E18	128,000 × 64	8,192,000	128,000	23,366.69
E19 pilot	23,895 × 128	3,058,560	23,895	10,225.84
E19 funded	128,000 × 128	16,384,000	127,000	54,419.81

The funded run completes its full schedule without a time-limit stop or optimizer reset; its final resumable snapshot is step 128,000, while held-out evaluation uses the best-validation step 127,000. Best weighted validation CE is 0.133244. The complete remote pipeline takes 55,700.91 s, including preprocessing, evaluation, benchmarking and transfer but excluding rental startup and the final cleanup wait. All 17 downloaded report/artifact files match their completion hashes. Source revision is `80dd445e5d17f54f53687a88459e011de4f9006f`; the generator SHA-256 and full result extract are retained in the repository's E19 report. Full-state snapshots preserve optimizer and random state; this uninterrupted run is not a test of cross-backend exact recovery.

Funded validation solves 70.90% of 1,000 boards. On easy/medium/hard tests, blank-cell accuracy is 99.69 / 97.12 / 86.99% and givens preservation is 100 / 100 / 99.60%; full-depth codec validation reconstruction is separately 100%. Wilson 95% intervals for the solved rates are 97.40–98.61 / 81.71–84.97 / 38.17–42.47%. These assume independent test boards, exclude training-seed variability and do not establish paired significance against prior runs. Per-board prediction archives were not retained. E20 supplies a second training seed below.

7.2.2 E20: a second training seed

E20 repeats the E19-funded scientific recipe with configured training seed 1 instead of 0. Both complete 128,000 updates at batch 128, using the frozen E6 codec, 200,000 base puzzles plus 8M cached unique puzzles, corruption 0.50⟶0.10, learning rate 0.0006 and 1,000-step warmup.

Shared codec and corpus input-manifest hashes match. The configured seed affects initialization, minibatch order, dropout and corruption; internal generator streams use that seed plus one. These influences are not separately controlled.

The run name, output path and operational cap also change. E20′s cap is 960 rather than 1,440 minutes; neither run reaches its cap. E20 records no optimizer reset or truncation, and its final full-state snapshot and selected best checkpoint are both step 128,000. Training-loop time is 54,122.43 s; pipeline time is 55,543.62 s under the E19 timing boundary. Best weighted code CE is 0.142558, compared with 0.133244 for seed 0. Separate physical A40 allocations and missing remote source inventories prevent a strict same-host causal attribution.

Table 6: Two-seed held-out aggregates. Each seed reports the same 2,000-board benchmark per difficulty; the mean is descriptive, not a pooled independent sample.

Quantity	Easy	Medium	Hard
Seed 0 solved boards	1,962	1,668	806
Seed 1 solved boards	1,945	1,592	665
Seed 1 solved rate	97.25%	79.60%	33.25%
Seed 1 minus seed 0	−0.85 pp	−3.80 pp	−7.05 pp
Two-seed mean	97.675%	81.500%	36.775%
Seed 1 Wilson 95% interval	96.44–97.88%	77.78–81.31%	31.22–35.35%

Both seeds retain strong easy-board performance and substantially weaker hard performance. E19-funded remains the best observed seed; neither two-seed means nor board-level Wilson intervals reliably estimate the population distribution of training outcomes. The intervals also exclude related-board dependence and repeated benchmark-guided recipe selection. Without per-board predictions, paired significance and agreement between the solved-board sets cannot be computed. No additional model evaluation is included in this version.

E20 validation solves 71.00%, compared with 70.90% for E19-funded, while hard blank-cell accuracy falls from 86.99% to 84.92%. Similar validation solving therefore does not imply similar external hard-set performance. The saved hard aggregates identify 670 valid grids, of which 665 preserve the givens: 1,330 failures are invalid grids and five additional failures are valid grids contradicting clues. Fourteen boards violate givens overall. Clue preservation alone cannot repair the dominant invalid-grid failure; the aggregates do not identify its internal cause.

Audit boundary. All 15 retained E20 completion files match recorded hashes; inference tensors equal the saved best-model tensors, and optimizer/training steps agree at 128,000. Generator SHA-256 is `b6ddadb7278e7f1c5542eecef676840f39cb829e802d189b65f2dadd645c2b3d`; the declared source is `4ad3646189197fe1ca76e45308848793fefe9a3f`. These checks establish retained-byte integrity and internal consistency, not independent reproduction. No checkpoint-bound ordered scoring manifest, per-board predictions or remote source digest inventory was retained. Current local board files match the published hashes, but they were absent from the remote transfer-verification manifest. Actual remote evaluation identity, complete duplicate-free coverage and exact source bytes remain independently unverified. Reported outcomes are historical aggregates with these limits.

E20′s recorded synchronized benchmark uses temperature 1, three warmups and 20 iterations. Batch 1/16/64/256 p50 is 29.453/48.506/174.326/581.791 ms; p95 is 30.323/49.274/175.352/583.397 ms. The CUDA GPU-load log is empty because the wrapper used a Metal-only probe. Contention at benchmark time is unknown, and these timings neither measure greedy quality generation nor establish a seed-dependent speed effect.

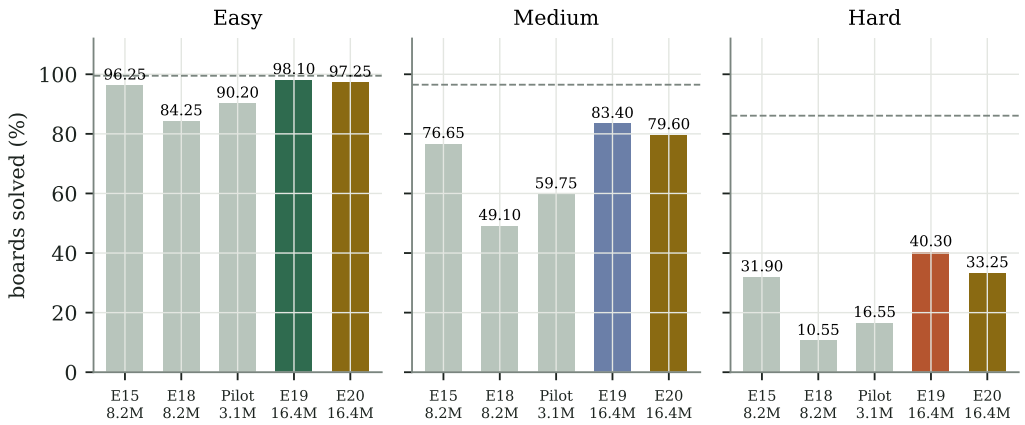


Figure 3: Completed MLX implementation recipes, with rounded generator presentations beneath each run; exact exposures are in the tables. Each bar reports 2,000 held-out boards; retained aggregate and provenance limitations apply. Dashed lines show Original NCM (Kumbong et al.), with its larger generator and 51.2M presentations; they are not matched-budget targets.

Funded synchronized generation at temperature 1.0 uses three warmups and 20 timed iterations, 12 generator forwards and completion through a host copy. Batch 1/16/64/256 p50 latency is 26.477 / 47.905 / 174.279 / 583.670 ms; p95 is 26.690 / 48.308 / 174.552 / 584.715 ms. This timing does not measure greedy quality decoding. Preflight records zero GPU utilization, zero allocated memory and no compute applications; an immediate pre-benchmark contention sample is not retained, so these are not certified idle-device limits. Neither cross-host A40 timings nor RTX 3080 comparisons isolate a hardware speed ratio.

The first funded allocation fails during shell preflight, before any training step, and is deleted. The failure is SIGPIPE from an early-closing version-check pipeline under `pipefail`; three regression cases reproduce it, and the corrected fresh attempt passes all 78 tests. Image startup timeouts recover before readiness. These are setup incidents, not extra training runs or reset-state continuations.

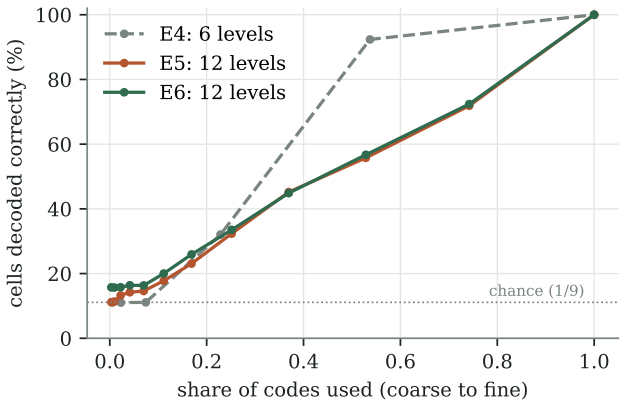


Figure 4: Cell accuracy when decoding from the first k levels of the codec, against the share of codes used. E4 stores almost all information in one level; E5 and E6 refine gradually.

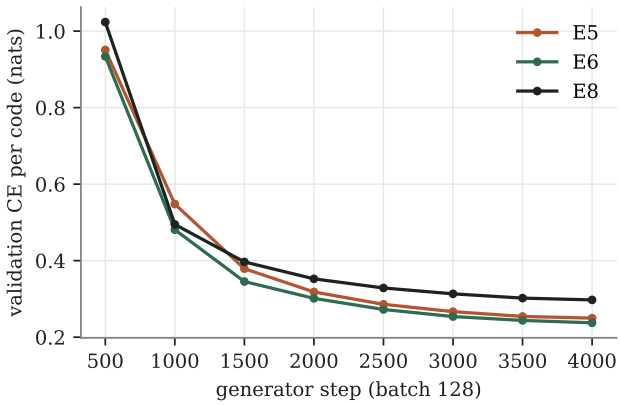


Figure 5: Validation cross-entropy per code during generator training.

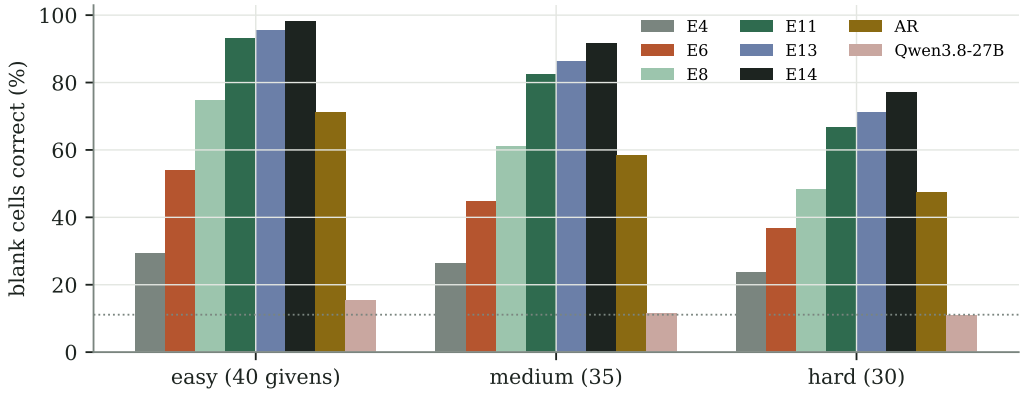


Figure 6: Blank cells filled with the reference digit, per test set. Chance is about 11%.

Why per-cell gains do not become solved boards. A board is about 300 codes. Teacher-forced accuracy per level in E6 ranges from 25% (level 0, which the puzzle barely determines) to 99.6% (the last level); generating greedily, an error at an early level changes the inputs of every later level. E8′s corruption training attacks exactly this compounding. Even 98% accuracy per cell, with independent errors, would solve only $0.98^{41} \approx 44\%$ of easy boards, which have 41 blanks. The paper's generator was trained on about 51 million sequences; E6 saw 0.5 million.

7.3 A large chat model as reference

Qwen3.8 27B (4-bit, served locally) answered each board in one request at temperature 0 with reasoning disabled. It solved none of 150 boards; its blank-cell accuracy was 11–15%, near chance, and about half its replies were not a 9×9 grid. With reasoning enabled and up to 6,000 tokens per board it did not finish ten boards in 95 minutes on the shared GPU. This is a reference point, not a matched comparison: the model is about 4,000 times larger and was not trained on the task.

8 The journey: experiment by experiment

> **In short.** The experiments examine architecture, training and data choices; some combine multiple changes. Experiment numbers (E0, E1, …) identify runs; the full record with every number is in `docs/EXPERIMENTS.md`.

Table 7: Experiments in order. Sudoku results are for the easy test boards; E21 onwards use TinyStories and report external generative perplexity (GPT-2 large, lower is better).

Run	Question	Change from the previous run	Result	What we learned
E0	Does the pipeline work end to end?	First version, 4,000 TinyWorld sentences	94.6% valid; overfits	The codec rebuilds 99.4% of sentences; models need more data
E1	NCM vs AR, fairly	12,000 sentences, best-validation checkpoints	NCM 89.8% vs AR 98.6% valid; 4–5× faster	Speed holds on a laptop; the accuracy gap is real
E2	Which level causes errors?	Force true codes for levels 0..k	Errors at every level	Not only the finest level is at fault
E3	Does reading coarser levels help?	Block-causal attention	92.2% valid, 1.5–1.9× slower	Small gain; kept as an option
E4	Does NCM solve Sudoku?	Puzzle as prefix, 6 levels	0% solved; AR 13.8%	One level holds all information: no real coarse-to-fine
E5	Does upstream's level schedule fix it?	12 levels × 16 codes, lookup encoder	0%; blank cells 52% (was 29%)	Gradual refinement, but level 0 is dead
E6	Does a learned level 0 fix it?	Learned level-0 sampler, GroupNorm, lower codec lr	0%; blank cells 54%	Level 0 now used; small gains; diverges at lr 1e-3
E7	Upstream's generator width	8 × 512	stopped early	Recipe search showed corruption was missing; GPU time moved to E8
E8	Does conditioning corruption help?	Uniform corruption 0.30 → 0.05	6.2% solved; blank cells 74.7%	The largest single gain: first solved boards; beats AR on blank cells; validation loss gets worse while generation gets better
E9	Does corruption from the model's own samples help more?	Replacements drawn from the generator	3.0% solved; blank cells 74.7%	No: same blank cells, fewer solved boards, 1.56× training time

Run	Question	Change from the previous run	Result	What we learned
E10	Upstream's width	8 × 512, lr 3e-4, 1.5M sequences, on the Mac	55.9% / 18.4% / 2.0%	Width alone does not help at this budget: matches the 256-wide E11
E11	More data	E8 on 2M sequences (4×), RTX 3080	55.2% easy, 18.1% medium, 1.6% hard solved	Data was the largest remaining lever; NCM now beats the same-size AR model
E12	Upstream's width on a rented GPU	8 × 512, stronger corruption, 3.25M sequences, NVIDIA A40	74.6% / 35.2% / 6.0%	Width adds little; data still dominates
E13	Stronger corruption	0.50 → 0.10 at E11′s budget	70.9% / 31.2% / 4.0%	Corruption strength matters: +16 points on easy
E14	Even more data	E11 on 8.2M sequences (16× E8)	87.4% / 53.4% / 11.2%	Data keeps paying; still not saturated
E16	Fresh unique puzzles vs repetition	E13 on 2M greedy-unique puzzles seen once	57.6% / 21.8% / 2.2%	Worse than E13: how puzzles are built matters more than uniqueness
E17	Unique puzzles built like the test boards	E13 on random-removal puzzles that happen to be unique; weights-only recovery	71.25% / 31.75% / 3.70%	Close to E13; optimizer reset limits interpretation
E15	Both levers + unique-solution puzzles	E14 budget, corruption 0.50 → 0.10, +4M unique puzzles	96.2% / 76.6% / 31.9%	Within 3.3 points on easy; roughly one quarter of upstream's generator size and one sixth of generator exposure
E18	Both levers, base data only	E14 budget, corruption 0.50 → 0.10	84.25% / 49.10% / 10.55%	Below E14 and E15; stronger corruption alone is not sufficient
E19 pilot	Larger-corpus short schedule	+8M unique puzzles; 3.059M presentations, batch 128, A40	90.20% / 59.75% / 16.55%	Below E15 at less exposure; not a matched comparison
E19 funded	Larger corpus and longer schedule	Fresh initialization; 16.384M presentations, batch 128, A40	98.10% / 83.40% / 40.30%	Best observed seed at twice E15 exposure; still below Original NCM
E20	Does the funded recipe repeat?	Fresh seed 1, same scientific settings and exposure	97.25% / 79.60% / 33.25%	Easy/medium gains persist; hard result varies by 7.05 points; audit limits remain

Run	Question	Change from the previous run	Result	What we learned
E21	Can NCM be trained on BPE story text?	New codec on TinyStories windows; generator corruption 0.75 → 0.25; AR control	Reconstruction 99.816%; perplexity 47.897 (AR 14.676)	The pipeline runs end to end; free-running text is far behind AR
E22	Is that corruption too strong?	Corruption 0.30 → 0.05, same codec and exposure	Clean code CE 1.66515; perplexity 54.923	Lower clean loss, worse generated text
E23	What happens without corruption?	Corruption 0 → 0	Clean code CE 1.38995; perplexity 97.776	Best clean loss, worst generated text
E24	Does E21 repeat across generator seeds?	Generator and AR seed 1, same frozen codec	Clean code CE 2.24909; perplexity 45.568 (AR 14.311)	E21′s result repeats; one codec only
E22 / E23, seed 1	Does the ordering repeat?	Generator seed 1, same frozen codec	Code CE 1.66106 / 1.38638; perplexity 55.114 / 93.833	The inverted ordering repeats in both generator seeds

What the journey shows. The first Sudoku configuration failed for a reason no reconstruction test would reveal: its codec rebuilds boards perfectly yet places nearly all information in one level (the codec-refinement figure, E4), so the generator faces one large guess instead of a series of small ones. The fixes taken from upstream each helped a little and none was enough on its own. Reading the paper's appendix in full showed the two largest gaps: conditioning corruption and a training budget about a hundred times larger. Corruption alone (E8) produced the first solved boards and lifted blank-cell accuracy by 20 points on every test set. Its validation loss was *worse* than without corruption, a reminder that teacher-forced loss does not measure what generation needs. The TinyStories ablation of Section 10 shows the same dissociation on text: less corruption lowers clean loss and worsens generated text.

9 Released checkpoints in MLX: inference parity

> **In short.** The released TinyStories, Sudoku and XSum checkpoints of Kumbong et al. run in an independent, inference-only MLX port. In the tested computations the port agrees with upstream PyTorch on CPU, up to stated floating-point effects, and the XSum model scores ROUGE within 0.1 of the quoted values. This section concerns upstream-trained weights. It does not reproduce upstream training and says nothing about the models we trained ourselves.

Throughout this section the models are called the **Released TinyStories checkpoint (Kumbong et al.)** and the released Sudoku checkpoints. They were trained by the original authors; we only convert and run them.

9.1 What was ported and how it is converted

The port (`ncm_mlx/official/`) implements the inference computation of the released codec and generator for TinyStories (unconditional BPE text) and for the easy, medium and hard Sudoku models. It is written from the published architecture and the released configurations and contains no upstream source code. It is pinned to upstream code revision `c70274273f489abc73832b2b98584a8483b6b203` and checkpoint revision `d342ce8aaea58830eef3b8ad473664673e41f692`. The weights are downloaded from their Hugging Face repository on demand and are not redistributed in our code repository.

Conversion does not need PyTorch. A small reader parses the checkpoint's zip container, tensor storage and reconstruction metadata through a restricted unpickler that accepts only the globals a tensor checkpoint uses and refuses all others; the tokenizer pickles are read by an unpickler that refuses every global. Each upstream tensor name must match exactly one explicit rule: it becomes an MLX parameter with a stated layout change (for example convolution weights change from `[out, in, kernel]` to `[out, kernel, in]`), or it is dropped. Dropped names are the EMA training state and two derived buffers, the level identifiers and the block-diagonal mask, which are first checked against the values the MLX model assumes. An unmatched name, a missing parameter or a shape mismatch stops the conversion, and a configuration whose SHA-256 differs from the pinned one is refused. The output is safetensors plus an architecture specification, tokenizer JSON and, for TinyStories, the level-0 distribution. This is a format and layout conversion, not quantisation or further training.

TinyStories uses the released 8,192-token BPE tokenizer. Replacing it with a similar tokenizer would change every input, so we reimplemented its pre-tokenisation, byte handling, merges and special tokens. Thirteen retained cases, including Unicode, agree with upstream in pre-tokenisation, token identifiers and round trip. Rare whitespace classes are not exhausted and remain a documented limitation.

9.2 Parity with upstream PyTorch

A parity script runs upstream PyTorch 2.15.0 on CPU and the MLX port (0.32.3) on `mx.cpu` in one process with identical inputs, and routes teacher-forced codes through each implementation's own generation loop. The TinyStories check uses four input texts for the codec and eight greedy samples; the Sudoku check uses the first 64 published easy boards.

Table 8: MLX port versus upstream PyTorch, CPU, fp32 unless stated.

Check	Released TinyStories	Released Sudoku-easy
BPE pre-tokenisation, identifiers, round trip (13 cases)	identical	not applicable
Codec embedding, largest absolute difference	2.9e-6	1.4e-6
Codec codes from real inputs, agreement per level	97.45–100%	100% (64 boards)
Per-level logits, largest absolute difference	0.100 (logit magnitudes up to 853)	7.2e-5 (magnitudes up to 29)
Same, both sides in exact fp64	3.2e-10	1.2e-13
Greedy generation, codes and decoded text	identical, 8 of 8	identical, 64 of 64

The TinyStories fp32 difference exceeds a naive absolute threshold of 10^{-4} for a numerical reason. The model carries activations of magnitude several hundred, and its codebook-tied output head subtracts large squared-codebook terms, which amplifies float32 rounding. Upstream PyTorch's own fp32 result is 0.071 from an exact-fp64 reference, and the MLX fp32 result is 0.126 from it. The exact-fp64 row shows that the two implementations compute the same function. The argmax agrees at every compared position.

The fp64 row is a diagnostic computation, run only for this check. Upstream normalisation layers, tied-head norm buffers and rotary tables, which it computes in float32 even for float64 inputs, are recomputed in float64. On the MLX side, composed operations replace primitives that do not preserve float64 accuracy on CPU, with `exp` and `erf` evaluated on the host. Production inference stays float32. This supports agreement of the tested computations; it does not establish bitwise identity of production inference. Apple GPUs also compute fp32 matrix products at reduced precision (Section 6.2), so Metal outputs can differ from CPU outputs at near-ties.

The four TinyStories codec inputs show per-level code agreement below 100% for a precise reason. The first differing code of each affected input is a near-tie in quantiser distance (gaps of 4.5e-3 and 2.4e-4 at distances near 10), and the changed code alters the residual and therefore

later levels. A repository test requires codes to match up to the first divergence and the two candidates to lie within a small relative distance. With four inputs this is a spot check and not a distributional claim.

9.3 Released Sudoku evaluation

On the published 2,000-board test sets, greedy decoding with the MLX port on the Metal GPU solves 99.40%, 96.30% and 86.20% of easy, medium and hard boards (1,988, 1,926 and 1,724 boards). The original paper reports 99.5%, 96.5% and 86.1% (Kumbong et al. 2026, Table 3). On the easy set, upstream PyTorch on this machine's CPU produces exactly the same 2,000 outputs, and Metal and CPU MLX outputs are also identical, so the 0.10-point difference from the quoted value does not come from the port. Upstream was not rerun locally on the medium and hard sets, so the differences of −0.20 and +0.10 points there are not diagnosed. These are results of upstream-trained models. They are separate from E19 and E20 and say nothing about our smaller generators.

9.4 Released XSum summarisation

The same port runs the released XSum generator and codec of Kumbong et al., which write a one-sentence summary of a news article (Narayan et al. 2018). This is the one conditional task in this paper, and it concerns upstream-trained weights: we neither train nor evaluate our own models on it. The article's first 512 GPT-2 token ids are the prefix. Level 0 comes from four learned queries. The remaining eight levels are decoded greedily (temperature 1, top-k 1), as in the released configuration, and a token readout with a frequency penalty and adjacency rules turns the codes into text. These rules follow upstream's evaluation script; the reference functions used in the parity check are extracted from upstream and not copied into our code. Conversion is strict in the same way as in Section 9.1 and checks the pinned checkpoint, tokeniser and test-set revisions.

Against upstream PyTorch on CPU, on eight fixed inputs (four test articles and four raw texts), GPT-2 encoding (11 cases, 500 references) and decoding (201 id rows) are identical; free-running greedy codes at all nine levels are identical; and the penalised readout gives identical ids and texts for 8 of 8 inputs. Teacher-forced per-level logits differ by at most 6.7e-4 in fp32, at logit magnitudes up to 1,118, and by 6.8e-13 in exact fp64, the same pattern as the TinyStories checkpoint of Section 9.2. Classifier-free guidance at scale 1.0 also yields identical codes.

Table 9: ROUGE F-measures (×100, stemmer on) of the released XSum model in the MLX port, greedy decoding, on the 11,334-article test set. Rows marked quoted are the values reported by Kumbong et al. (Kumbong et al. 2026, Sec. 4.3 and Table 2), not measured here.

Setting	ROUGE-1	ROUGE-2	ROUGE-L
MLX port, released defaults, no guidance	34.93	12.71	27.94
Quoted, no guidance ("8 NFE")	35.0	12.8	28.0
MLX port, guidance scale 1.0	35.15	12.61	27.98
Quoted, with guidance ("16 NFE")	35.3	13.0	28.2
MLX port, guidance scale 0.5	35.20	12.83	28.11

The unguided full-set scores are within 0.1 of the quoted values on all three measures. The paper reports its guided row at guidance weight 0.5 (Kumbong et al. 2026, App. F.4, Fig. 14), where its ROUGE curves peak. At scale 0.5 (assuming upstream's `cfg_scale` is that weight) the port is within 0.2 of the quoted guided scores on all three measures; at 1.0 it is lower on ROUGE-2 and ROUGE-L. The paper counts generator calls per output, 8 for the unguided model and two per guided step; our port makes 9 generator forwards (a level-0 query pass and eight levels), so the paper apparently does not count the level-0 pass, which we have not confirmed. ROUGE here is computed with the `rouge-score` package at version 0.1.2, whose version upstream does not pin (Lin 2004). On a 200-article random sample, Metal and CPU produce the same summary for 181 articles and ROUGE-1/2/L of 34.51/12.72/27.55 and 34.63/12.64/27.50; Metal's reduced-precision matrix products flip near-ties. The released defaults run at 36.1 articles per second at batch 16 on the Apple M5 Pro (a busy desktop, 200 articles, excluding scoring).

What this does not show: the summaries are fluent-looking but often contain broken words and wrong facts (one reads "won herptptathlon" for a heptathlon result), and ROUGE overlap measures neither faithfulness nor quality. Only greedy decoding is compared with upstream, and parity was run on CPU only. Minimum-Bayes-risk decoding and sampled summaries are not implemented or checked. The penalty readout changes every parity text but not ROUGE at 200 articles, which deserves a larger ablation. The result shows that a released conditional checkpoint runs in our port and reproduces its published scores closely; it does not extend to our trained models, which are all unconditional.

9.5 Latency, and what this section does not establish

The recorded eager TinyStories batch-1 median is 39.8 ms on the Metal GPU for the MLX port and 280 ms for upstream PyTorch on CPU, each completing a 128-position sample through a host copy. MLX used three warmups and 20 timed iterations, PyTorch two and ten; the GPU was shared with desktop applications. This compares a working GPU port with a CPU reference, because the tested PyTorch MPS path rejects the non-divisible area interpolation the model requires. It is not an architecture-only speed ratio. Later repository work adds prefix caching and optional compilation for the port; Section 10.8 reports their timings and the output differences they introduce, and no quality score here is attached to those modes.

The port establishes that the tested inference computation is implemented independently and checkably. It does not reproduce upstream training or its results. Upstream's TinyStories generative-perplexity protocol is not reproduced: Section 10 scores the Released TinyStories checkpoint with our own pinned scorer. Sampled outputs use MLX's random generator and so do not match upstream sample for sample. The LM1B, OpenWebText and WMT14 checkpoints are not converted; the XSum generator is (Section 9.4). A fresh-environment installation and inference check on an accelerator remains a verification limit.

10 Controlled text: TinyStories pilots

> **In short.** We trained a small NCM and a small AR baseline on TinyStories, a synthetic children's-story corpus, and compared them under one fixed external scorer. The NCM reconstructs held-out windows accurately but generates text that scores far worse than the AR baseline. Varying the conditioning-corruption schedule moves clean teacher-forced loss and free-running quality in opposite directions, in two generator seeds that share one frozen codec. The comparison does not match total compute, uses synthetic prose and includes no conditional task.

10.1 Scope and labels

Three kinds of model appear. **MLX NCM** means the NCM models trained in this project (called "MLX implementation" in the Sudoku tables). **MLX AR baseline** is the autoregressive model trained here on the same windows. **Released TinyStories checkpoint (Kumbong et al.)** is the original authors' released model, run through the port of Section 9; it is a reference and is not trained here. A fourth row, *held-out data*, scores real held-out stories windowed in the same way. The study is unconditional generation of 128-token story openings from a null prefix. No supplied prompt, continuation or other conditional task is evaluated.

10.2 Representation and training contract

Table 10: TinyStories contract shared by the MLX NCM and MLX AR baseline runs. These differ from the character and Sudoku path of Sections 5–7.

Component	Setting
Corpus	`roneneldan/TinyStories` (Eldan and Li 2023), V2 GPT-4 train and validation text, revision `f54c09fd23315a6f9c86f9dc80f725de7d8f9c64`. The pinned dataset card identifies it as GPT-4-generated and lists CDLA-Sharing-1.0.

Component	Setting
Splits	2,717,495 training stories. The validation file is split by SHA-1 parity of the stripped story into 13,960 validation and 13,670 test stories, so splits are document-level; document overlap with training is checked in preprocessing.
Tokenisation	The released 8,192-token BPE (EOS 0, PAD 1). Each story is taken verbatim with its leading newline; the window is the first 128 tokens of story plus EOS, then right padding. Later parts of long stories are dropped; about 5% of training windows contain their story's EOS and about 0.38% of training positions are padding. Losses include padding.
Hierarchy	12 levels with lengths 1, 8, 16, 24, 32, 40, 52, 64, 76, 92, 108, 128 and 2,048 codes per level.
Generation	Level 0 is sampled from the stored empirical unigram of level-0 codes. Exactly 11 generator forwards predict the remaining levels, and the result is decoded to 128 tokens.
Conditioning	A single null EOS prefix. Previous concept levels supply the only internal conditioning.
Codec	Token and position lookup encoder, GroupNorm (8 groups), learned strided level-0 cascade, area pooling and linear interpolation at other levels, per-level residual transforms and one transformer decoder block; width 256, 4 heads. 15,508,096 saved values, of which 6,291,456 are EMA codebook values and 9,216,640 are gradient-trained.
MLX NCM generator	6 blocks, width 384, 6 heads, QK normalisation, RoPE base 100,000, codebook-tied distance heads, block-diagonal level attention. 11,832,587 values.
MLX AR baseline	The same 6-block, width-384, 6-head stack with a tied 8,192-token embedding: 13,794,048 parameters, 128 cached next-token forwards.
Released TinyStories checkpoint	36,434,816 codec values and 88,605,707 generator values (Section 9).
Optimisation	AdamW, peak learning rate 5e-4 with cosine decay to 5e-5, betas (0.9, 0.95), gradient clipping 1, compiled steps. Codec: warmup 1,000, weight decay 0.01, dropout 0.3, fine-level dropout 0.5, commitment 0.25, EMA decay 0.95, dead-code restart below count 0.01. Generator and AR: warmup 500, weight decay 0.1, dropout 0.1. Generator loss uses $\alpha = 0$ (equal weight per code). Final exported weights are evaluated; no checkpoint is selected on validation data.
Stage budgets	Codec: 14,000 updates $\times$ 256 = 3,584,000 windows, trained once with seed 0 and then frozen. Generator: 15,000 $\times$ 64 = 960,000 windows (122,880,000 output-token positions) per run. AR: 7,500 $\times$ 128 = 960,000 windows per run.

Seed 0 training-loop times on the Mac were 5,517.25 s for the codec, 13,898.17 s for the strong-corruption generator, 14,435.59 s for the reduced-corruption generator, 20,734.07 s for the no-corruption generator and 2,062.03 s for the AR baseline, none time-limited. These are

elapsed times on a desktop with other workloads, not FLOP, energy or cost measurements; the longer no-corruption time cannot be attributed to corruption alone. The seed-1 generators took 18,842.73 s (strong), 22,419.19 s (reduced) and 18,706.68 s (none) under similarly uncontrolled desktop load; the seed-1 AR training time is not reported. Configured seeds are offset internally (the generator derives initialisation and minibatch streams from seed + 1 and the AR from seed + 2), so configured seed 0 does not mean every stream is seeded with zero.

What the budgets do and do not match. The generator and AR runs see the same number of windows (960,000) and use the same block stack. Total compute and total system size differ. The NCM's teacher-forced input has 641 positions per window (640 predicted code positions plus the EOS prefix) against 128 for the AR; sequence length alone is not a FLOP estimate. The NCM also needs a codec, trained separately at additional cost and counted here as 15,508,096 more values, so the MLX NCM system holds 27,340,683 saved values against 13,794,048 for the AR baseline. The Released TinyStories checkpoint has a much larger parameter count and training budget and is not a matched reproduction target.

10.3 Evaluation protocol

Three different quantities are reported and must not be conflated.

1. **Reconstruction.** Non-padding token accuracy and exact-window rate when held-out windows are encoded and decoded through the full hierarchy, on the first 4,096 ordered held-out test windows.
2. **Clean code CE.** The generator's position-weighted cross-entropy in nats per code on the same windows when every earlier level is supplied correctly. Values from different codecs, such as the released model's, are not directly comparable with ours.
3. **Free-running quality.** Each model generates 256 outputs for each of five sampling seeds (42, 0, 1, 2, 3), 1,280 outputs per model, at batch 128, temperature 1, top-k 0 and top-p 1. The outputs are scored by the external model GPT-2 large at pinned revision `32b71b1`. Cleanup stops each output at the first EOS and removes PAD tokens before decoding. The scorer adds no beginning-of-sequence token, so every token after the first is scored; padded targets are masked. NLL sums are divided by scored-token counts within each sampling seed, and the five resulting perplexities are averaged. Empty or one-token strings would be unscorable and counted separately; none occurred in any model or run. The scorer's 1,024-token limit was never reached (retained records have at most 155 scored tokens).

Generative perplexity is a fixed external probability score of generated text. It is not the generating model's own perplexity, and it is neither a human rating nor an LLM-judge score. It does not directly establish coherence, grammaticality, truthfulness or usefulness. The five sampling seeds are repeated draws from one trained model; the standard deviation across them is not a confidence interval over training runs. Temperature, top-k and top-p sensitivity was not evaluated. Novelty is checked only by exact match of the first 32 output tokens against training windows, and by the diagnostics below.

10.4 The corruption ablation, with two generator seeds

During generator training, a fraction p_k of the conditioning codes of level k is replaced by random codes of that level (Section 5.2), falling linearly across levels from p_{first} to p_{last}. We compare three schedules with fresh generators on the same frozen codec, the same level-0 unigram, tokenizer, architecture, optimizer and window exposure: **strong** $0.75 \rightarrow 0.25$ (E21, repeated as E24), **reduced** $0.30 \rightarrow 0.05$ (E22) and **none** $0 \rightarrow 0$ (E23).

Every generator schedule was run with configured generator seed 0 and again with seed 1. Each seed-1 configuration differs from its seed-0 counterpart only in the configured seed, run name and output path. Both generator seeds use **the same frozen seed-0 codec**, so the repeat measures generator variability conditional on one codec and not variability of the full two-stage procedure. The AR baseline was trained once for seed 0 and once for seed 1 (in the E24 run); within each seed it is a single control shared by all three schedules, not three independent runs. All completed outcomes are reported, and none was replaced by a selected winner. The E24 controller was interrupted once, by a disk-headroom guard after generator training had finished; the original queue then resumed at AR training without retraining the generator or changing a recipe.

Table 11: Corruption ablation, MLX NCM versus controls. Perplexity is external GPT-2-large generative perplexity, the mean of five sampling-seed values (their standard deviation in brackets), lower is better. Non-padding reconstruction is 99.816% for every MLX NCM row because the codec is shared.

Model	Generator seed	Corruption	Clean code CE (nats/code)	Generative perplexity
MLX NCM, strong (E21 / E24)	0	0.75 → 0.25	2.24875	47.897 (0.797)
	1	0.75 → 0.25	2.24909	45.568 (0.909)
MLX NCM, reduced (E22)	0	0.30 → 0.05	1.66515	54.923 (1.548)
	1	0.30 → 0.05	1.66106	55.114 (1.568)
MLX NCM, none (E23)	0	0 → 0	1.38995	97.776 (2.107)
	1	0 → 0	1.38638	93.833 (1.424)
MLX AR baseline	0	not applicable	not applicable	14.676 (0.450)
	1	not applicable	not applicable	14.311 (0.237)
Released TinyStories checkpoint (Kumbong et al.)	not trained here	not applicable	1.37032 (different codec)	16.259 (0.315)
Held-out data	not applicable	not applicable	not applicable	8.529 (0.067)

The two measurements order the schedules oppositely, in both generator seeds. Clean code CE falls as corruption falls: 2.24875, 1.66515 and 1.38995 for seed 0 and 2.24909, 1.66106 and 1.38638 for seed 1. Free-running perplexity rises: 47.897, 54.923 and 97.776 for seed 0 and 45.568, 55.114 and 93.833 for seed 1. Between the two generator seeds the clean code CE of a schedule changes by at most 0.005 nats per code, while perplexity changes by −2.329 (strong), +0.191 (reduced) and −3.943 (none), within a few sampling-seed standard deviations (0.8 to 2.1). The ordering of the three schedules is the same in both seeds, but two generator seeds are too few to estimate how much a single schedule varies across training runs. The best MLX NCM configuration scores about 3.3 times (seed 0) and 3.2 times (seed 1) the AR baseline's perplexity. A plausible reading is that a generator trained only on correct context is more sensitive to its own earlier errors, which corruption is meant to address. The experiment does not isolate that mechanism: corruption strength and schedule, optimisation and level-wise error behaviour are not separated, and no control with matched total corruption but different distribution across levels is reported here.

Each seed-1 evaluation report passes the result audit built into the evaluation pipeline (2,560 scored outputs per report). The independent recomputation described below was carried out for seed 0 only. The seed-1 evidence has the same format as seed 0; both are attached to the paper release as `ncm-mlx-tinystories-evidence-e21-e24.tar.gz` (SHA-256 `4a744f5306ebef3433abbab9647372d6ad00949f616c7befe9476fbccb0d20b8`, download).

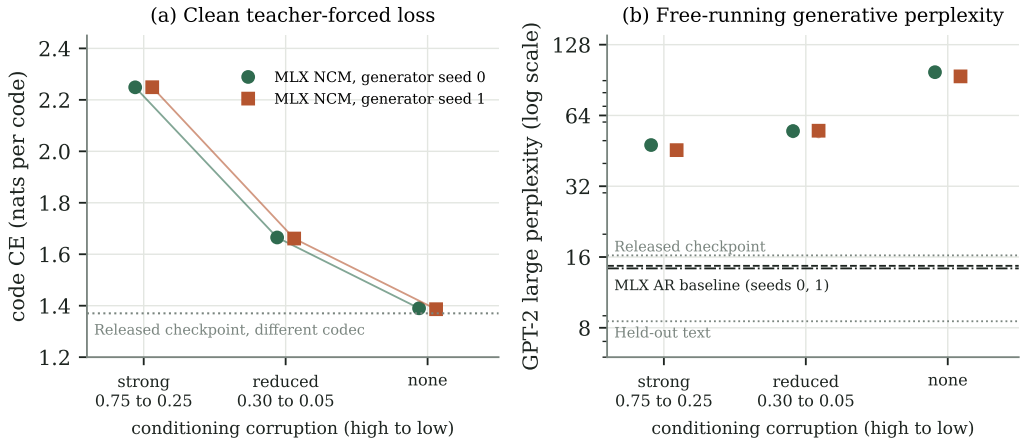


Figure 7: Corruption ablation on TinyStories, both generator seeds (circles: seed 0, squares: seed 1), one frozen codec. (a) Clean teacher-forced code cross-entropy of the MLX NCM; the Released TinyStories checkpoint (Kumbong et al.) uses a different codec. (b) GPT-2 large generative perplexity on a logarithmic axis: the five sampling-seed values of each model as small points and their mean as the marker, with horizontal lines for the MLX AR baseline (seeds 0 and 1), the Released TinyStories checkpoint and held-out text. Lower clean loss goes with worse free-running text. The two panels use different measures and are not on a shared axis.

10.5 Controls and diagnostics

Diagnostics on 1,280 outputs per model (the held-out-data row uses 1,280 real windows) help to read the perplexities, with the caveat that none is a quality measure.

Table 12: Output diagnostics. Seed-pair entries read generator seed 0 / generator seed 1. Tokens are BPE tokens after cleanup. Distinct-3 is the fraction of distinct 3-grams. Repeated 4-grams is the percentage of 4-grams repeated within an output. Opening copies are exact matches of the first 32 tokens to a training window.

Model	Mean output tokens	EOS inside window (%)	Distinct-3	Repeated 4-grams (%)	Opening copies (%)
MLX NCM, strong	127.36 / 127.27	7.11 / 6.95	0.4951 / 0.4787	0.51 / 0.63	0.00 / 0.00
MLX NCM, reduced	127.20 / 127.33	8.05 / 6.64	0.5374 / 0.5455	0.40 / 0.37	0.00 / 0.00
MLX NCM, none	127.14 / 127.21	7.50 / 6.48	0.6239 / 0.6151	0.19 / 0.22	0.00 / 0.08
MLX AR baseline	127.61 / 127.65	3.12 / 4.14	0.4910 / 0.4797	0.84 / 0.89	1.25 / 1.25
Released TinyStories checkpoint	128.00	0.31	0.4595	0.58	0.70
Held-out data	127.32	5.94	0.4850	1.01	4.61

Lengths are similar across models, so length alone does not explain the perplexity ordering. Higher distinct-3 and lower repetition for less-corrupted generators coexist with much worse perplexity: diverse but ungrammatical text can look good on these diagnostics, so they must not be read as quality. Few or no exact opening matches (none in five of the six MLX NCM runs, one output in 1,280 for the seed-1 no-corruption run) do not rule out memorisation; only the first 32 tokens are compared, and about 4.7% of test windows share their opening with a training

window, as do 4.61% of the sampled held-out-data windows. Substring copying and semantic near-duplicates are untested. The AR baseline's own held-out non-padding token cross-entropy is 1.32042 (seed 0) and 1.32396 (seed 1) nats per token; this is a third quantity, not comparable with code CE or with external perplexity. The Released TinyStories checkpoint reconstructs 99.981% of non-padding tokens and 92.969% of windows exactly, against 99.816% and 84.937% for our codec, on the same windows.

10.6 Sensitivity of the seed-0 scores to aggregation

To test whether the seed-0 ranking depends on how perplexity is summarised, a CPU-only audit verified 7,680 saved output texts against their scorer-record hashes and reproduced every reported mean to three decimals. It then reaggregated the same per-output NLLs, with no new generation or scoring. *Pooled token perplexity* exponentiates total NLL divided by total scored tokens. *Equal-sample geometric perplexity* exponentiates the mean per-output NLL per token, giving every output equal weight. *Median sample perplexity* is the median output's value.

Table 13: Seed-0 scores under four aggregations. These summarise one scorer and are not independent quality measurements.

Model	Reported mean of seed perplexities	Pooled token	Equal-sample geometric	Median sample
MLX NCM, strong	47.897	47.891	47.862	46.196
MLX NCM, reduced	54.923	54.906	54.868	52.635
MLX NCM, none	97.776	97.757	97.680	95.498
MLX AR baseline	14.676	14.671	14.708	13.612

The ordering strong < reduced < none, with the AR baseline lowest, holds under every aggregation. Mean scored lengths are similar (127.1, 127.1 and 127.5 GPT-2 tokens). Within outputs of 96–127 scorer tokens, pooled perplexity is 46.089, 53.804 and 94.413 for strong, reduced and none; within 128–159 tokens it is 50.807, 56.304 and 100.457, with at least 494 outputs per NCM model in each group, and the AR baseline remains lower in both. These length groups are post hoc and do not control every difference between models. The audit was not repeated for seed 1.

10.7 Generated examples

The examples are the first output (sampling seed 42, index 0) of each seed-0 model, not selected for fluency, shown as opening excerpts. Their complete text and token identifiers are retained with the evidence.

MLX NCM, strong corruption (E21): Once there was a little boy called Jack. Joe was was three years old and discover. One One day, he wanted to go and look for exploring.

MLX NCM, reduced corruption (E22): He and his mum were walking by the beach beach. T Tise was very happy because he wanted to learn how to try it. He wanted to show his plane plane and take off his boat.

MLX NCM, no corruption (E23): Jack and his mum were walking by the beach at. T T gears. Tom loved to harbor because learn learned he wanted to travel beyond and was and excited, green trees and trees.

An informal, unblinded reading of the first eight retained outputs of each run, performed with the assistance described under AI assistance, finds recognisable story vocabulary and short plausible phrases mixed with duplicated words, incomplete clauses, changing names and pronouns, implausible actions and unstable continuity. The first strong-corruption output switches between Jack, Joe and John, and the first no-corruption output shifts among a beach, trees, a park and several animals. This is a description of a small fixed sample, not a human rating, a blinded comparison or an error-rate estimate. Many outputs, including real held-out

windows, end mid-story because outputs are capped at 128 BPE tokens. This revision shows seed-0 examples only.

10.8 Latency: a quality/latency tradeoff

Table 14: Seed-0 recorded latency, median / 95th percentile in ms, batch 1, 16 and 64. NCM is the eager MLX NCM with strong corruption; the AR baseline uses a key-value cache. Timings use three warmups and 20 iterations and include a host copy of the decoded tokens.

Runner	Batch 1	Batch 16	Batch 64
MLX NCM, eager (11 generator forwards)	12.791 / 13.005	45.432 / 45.888	223.878 / 227.090
MLX AR baseline, KV cache (128 forwards)	81.841 / 83.411	142.712 / 144.585	357.411 / 365.647

In these recorded runs the NCM is faster than the AR baseline at all three batch sizes while generating much worse text under the external scorer, so this is a quality/latency tradeoff and not a matched-quality speedup. The timings come from a contended desktop: other services were active, and a zero reading of device utilisation before a benchmark cannot exclude load during it. The reduced- and no-corruption benchmarks show larger latency tails and nonzero utilisation at their boundaries, and their slower batch timings should not be read as an architectural effect of corruption. The NCM and AR timings use different sampler implementations, and the AR peak MLX allocation at batch 1 is about 62.6 MiB against about 184.6 MiB for the NCM.

Cached and cached-plus-compiled NCM modes have separate timings. In the tested cold runs, their sampled outputs differ from eager outputs at batches 16 and 64 for the strong-corruption model (changed-token fractions 0.049–0.317%) and for the no-corruption model (0.342–0.403%), and match for the reduced-corruption model; this is one seed per tested batch and mode. A cache-correctness result on fixed greedy fixtures does not remove this qualification, so the eager quality scores above are not attached to the optimised timings. Timings are reported for seed 0 only.

10.9 What these experiments establish and what remains open

The experiments establish that independently trained short-prose reconstruction and unconditional generation run end to end in MLX, and that reconstruction (99.816% of non-padding tokens) far exceeds free-running quality (generative perplexity roughly 3.2 times the AR baseline's or more). They show, in two generator seeds on one frozen codec, that lower clean teacher-forced code loss accompanied worse generated text under this scorer when conditioning corruption was reduced or removed.

They do not establish quality on naturally occurring documents; any conditional task, long-form coherence or instruction following; human-rated quality; the effect of sampling temperature or truncation; independence of the result from the codec (only one codec seed was trained); equal-total-compute behaviour; the ceiling of the architecture at the released model's far larger budget; or memorisation safety. No energy, monetary or independent-hardware measurement is available.

11 Scaling, data and precision: TinyStories follow-ups

> **In short.** After the corruption ablation of Section 10 we asked what else might narrow the gap between the MLX NCM and the MLX AR baseline. Training-side tweaks and sampling policies did not (Section 11.2). Making the generator larger and showing it more data did: the ratio of generative perplexities fell from 3.33 to 2.13 across the grid we ran (Section 11.4), with bf16 mixed-precision training used for the four-times-data runs (Section 11.3). A small run on web text (Section 11.5) only checks the pipeline. The ratio remained above two. Every row is a single

> training seed, except the 12M generator at four times the data, total compute is not matched, the codec is frozen from Section 10, the text is synthetic and the precision differs between some rows. We read the result as a trend under those conditions. It is not a scaling law and it does not predict what happens at a billion parameters.

11.1 Scope and labels

Everything here uses the setup of Section 10: the same frozen 15.5M-value codec (non-padding reconstruction 99.816%), tokeniser, 12-level hierarchy, optimiser, evaluation protocol and external scorer (GPT-2 large, 256 outputs for each of five sampling seeds). Unless stated, corruption is $0.75 \rightarrow 0.25$ and the loss weight is $\alpha = 0$. "12M", "50M" and "87M" are shorthand for MLX NCM generators of 11,832,587, 49,644,683 and 87,426,059 parameters (6×384, 12×576 and 12×768 blocks). Each size has its own MLX AR baseline built from the same blocks (13.8M, 52.6M and 91.3M parameters), trained on the same windows. They are our models. The Released TinyStories checkpoint of Kumbong et al. appears only as a reference. The AR baseline grows with the NCM generator, so the ratio compares the two methods at each size, not one fixed system. The numbers come from saved evaluation reports and are kept in `docs/research/tinystories-scaling.json`, with the experiment log entries E25–E32 in `docs/EXPERIMENTS.md`.

The runs were made on several machines: an Apple M5 Pro (the Mac, Section 10), an RTX 3080 and rented NVIDIA GPUs (an A40, an RTX A6000, an RTX PRO 6000 and an RTX 5090) through MLX's CUDA backend, all with MLX 0.32.3. Training time is therefore not compared across rows.

11.2 Follow-ups that did not narrow the gap

Matched-constant corruption (E25). E21′s corruption falls from 0.75 to 0.25 across levels, and its expected replacement count is 198.95 of 513 conditioning positions per window. A constant probability of 0.3878 reproduces that count. The matched-constant generator (seed 0) scores 47.052 (SD 1.245) with clean code CE 2.057, against E21′s 47.897 and 2.249 and E24′s 45.568. The difference is inside the 2.33 spread between E21 and E24, so we find no evidence that the taper itself matters at this amount of noise. Equal expected replacement is not equal semantic perturbation, because a replaced code at a coarse level changes much more of the text than one at a fine level. A second seed was queued and never run.

Stronger corruption and equal level loss (E28). Two further variants were trained with two generator seeds each. Raising corruption to $0.90 \rightarrow 0.50$ made things worse, as the clean code CE rises to about 2.84: perplexity 55.329 (seed 0) and 53.297 (seed 1), against 47.897 and 45.568 for E21 and E24. Giving every level equal loss weight ($\alpha = 1$, so the first predicted level carries one eleventh instead of 1.25% of the weight) is indistinguishable: 47.543 and 45.740. The NCM/AR ratios are 3.77 and 3.73 for stronger corruption and 3.24 and 3.20 for equal level loss, against 3.26 and 3.18 for E21 and E24. These four runs used gradient accumulation (2×32 windows per update) on a 12 GB card, which the public configurations do not express (see Code and reproducibility).

Sampling policies (E27). Perhaps the gap is partly a sampling problem. We swept temperature (global and per level), top-k, top-p and the level-0 distribution for the E21 and E24 checkpoints, and compared each NCM setting with the AR baseline at the same distinct-3 diversity, because lowering temperature lowers both models' perplexity. The ratio at matched diversity stayed between about 3.0 and 3.7 for every sampler, with the best one (cooling only levels 4 to 11 to temperature 0.5) at 3.06 against 3.29 at temperature 1 (E24: 2.99 against 3.25). A lower temperature drops E21 from 47.9 to 31.2 at that setting, which looks like a large gain, but the AR improves by the same factor for the same diversity cost. In log terms we attribute roughly 5 to 10% of the gap to sampling and the rest to the learned model, with an uncertainty of a few points. Sharpening the stored level-0 unigram changes nothing. The driver is `scripts/sampling_study.py` in the repository (see Code and reproducibility). A supporting diagnostic on 64 held-out windows (original target-code cross-entropy after the generator's own sampled prefix) is 11.26, 13.11 and 17.77 for E21, E22 and E23: the less corrupted generator copes worse with its own errors, which fits the opposite orderings of Section 10 without proving the mechanism.

11.3 Mixed precision: bf16 quality parity

The larger runs below use optional bf16 mixed precision (fp32 master weights and optimiser state; norms, tied level heads, rotary application and the loss in fp32; the codec trained in fp32 and never cast). To check that it does not change quality, we trained the E21 recipe in bf16 (E29, RTX 3080, seed 0, same frozen codec). Its clean code CE is 2.24705 and its generative perplexity 49.091 (SD 1.766), against fp32 values of 2.24875 and 47.897 (E21), 2.24909 and 45.568 (E24) and 2.24708 and 48.964 (the 12M fp32 run of Section 11.4, on a different GPU). The code CE differs from the three fp32 runs by at most 0.0021, and the perplexity is 0.13 above the highest of them, well inside its own sampling-seed SD of 1.77. On the same card, the generator training step took 0.157 s in fp32 and 0.104 s in bf16 (about 1.5 times faster) and peaked at 4.77 GB instead of 7.46 GB (36% less) at batch 32; the AR step was 1.53 times faster with 16% less memory. At batch 64 fp32 did not fit the 12 GB card.

This is a parity check at one size and one seed, against three fp32 values from different hardware. It supports using bf16 for the 12M recipe. It does not show parity at 50M or 87M, or at four times the data, where the bf16 runs below have no fp32 counterpart.

11.4 Size and data

Design. We trained three generator sizes at the original 960,000 windows in fp32 (E30: 15,000 updates at batch 64 for the generator and 7,500 at 128 for the AR), and again at four times the data, 3,840,000 windows (60,000 updates, AR 30,000), in bf16 (E31 and E32). The 12M run at four times the data was repeated with training seed 1. Every run uses the frozen E21 codec, so the codec is not retrained or enlarged with the generator, and a larger generator may be limited by it. The AR baselines also see four times the windows.

Table 15: Size and data grid. Perplexity is GPT-2 large generative perplexity, the mean of five sampling-seed values (SD in brackets), lower is better. Ratio is NCM over AR perplexity. Rows 1–3 are fp32 at 960,000 windows; rows 4–7 are bf16 at 3,840,000 windows. Training seed 0 except row 5.

#	Generator	Data	Clean code CE	NCM perplexity	AR perplexity	AR token CE	Ratio
1	12M	1×	2.24708	48.964 (1.236)	14.705 (0.260)	1.31481	3.33
2	50M	1×	1.90405	32.226 (0.609)	13.254 (0.232)	1.18791	2.43
3	87M	1×	1.80314	30.688 (0.985)	12.888 (0.446)	1.15069	2.38
4	12M	4×	2.01235	38.416 (1.316)	12.601 (0.264)	1.16798	3.05
5	12M, seed 1	4×	2.01741	36.186 (0.467)	12.525 (0.195)	1.16871	2.89
6	50M	4×	1.62073	26.148 (0.547)	11.838 (0.436)	1.04535	2.21
7	87M	4×	1.51620	24.404 (0.848)	11.484 (0.341)	1.01081	2.13

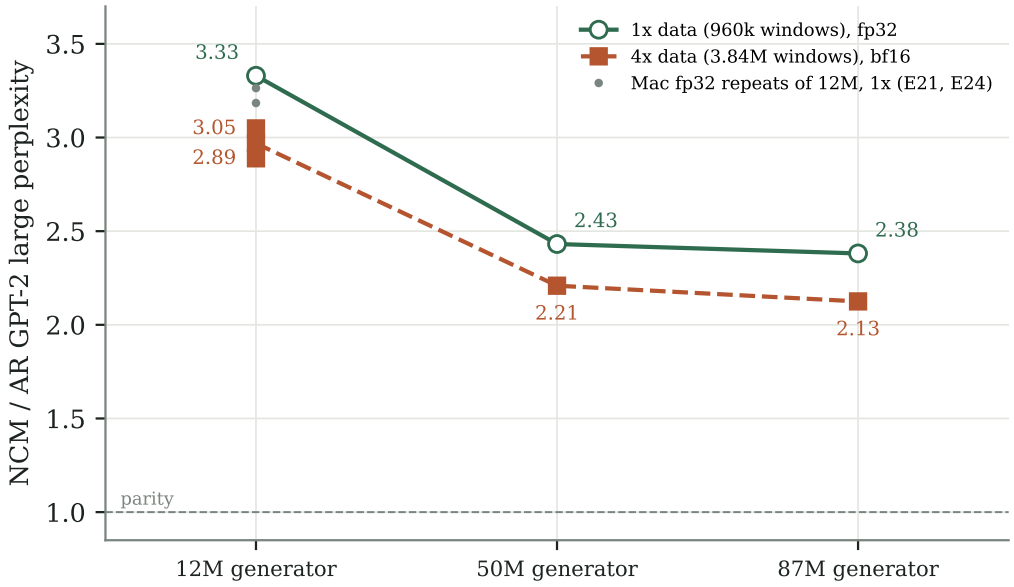


Figure 8: NCM over AR ratio of GPT-2 large generative perplexity by generator size, at 1× data (fp32) and 4× data (bf16). The 12M point at 4× is the mean of two seeds, shown individually; small dots are the Mac fp32 runs E21 and E24. One training seed per point otherwise; the two 12M seeds at 4× differ by more than the 50M and 87M points. The released checkpoint is not plotted, since no AR baseline here matches its budget. Compute is not matched, and precision and hardware differ between the two series.

What the grid shows.

- *Size, at fixed data.* From 12M to 50M the NCM perplexity falls by 34.2% (48.964 to 32.226) while the AR's falls by 9.9%, and the ratio from 3.33 to 2.43. From 50M to 87M the changes are 4.8% and 2.8% (ratio 2.43 to 2.38). At a fixed 960,000 windows, most of the gain arrives at the first step. The clean code CE keeps falling (2.247, 1.904, 1.803).
- *Data, at fixed size.* Four times the data lowers the NCM perplexity by 21.5% at 12M, 18.9% at 50M and 20.5% at 87M, and the AR's by 14.3%, 10.7% and 10.9% (against E21′s 47.897 instead of the 12M run of row 1, the 12M reduction is 19.8%). In relative terms, which is what the ratio measures, the NCM gains more from extra data than the AR does at every size, but at 12M the ratio only moves from 3.33 to 3.05 (seed 0) and 2.89 (seed 1).
- *Both together.* At four times the data the 50M and 87M generators reach ratios of 2.21 and 2.13 and the 87M NCM perplexity is 24.404, the best of our NCM models, against 11.484 for its AR baseline. That perplexity is 1.5 times the Released TinyStories checkpoint's 16.259 (a model with a much larger budget) and about 2.9 times the held-out text's 8.529.
- *The gap persists.* No point reaches parity, and the largest model still has about twice the AR's perplexity.

How much of this can be trusted. The two 12M seeds at four times the data differ by 2.230 perplexity (38.416 and 36.186) and the three 12M fp32 runs of Sections 10 and 11.4 differ by 3.4 (45.568 to 48.964), while the difference between the 50M and 87M perplexities is 1.54 (1×) and 1.74 (4×). The small size step from 50M to 87M is therefore not distinguishable from the seed and hardware variation measured at 12M; no repeat exists at the larger sizes. The first step from 12M to 50M (a 16.7-perplexity change at 1×) clearly is. The sampling-seed SDs (0.5 to 1.3) describe repeated draws from one model, not training-run variation. The 4× rows are bf16 and the 1× rows are fp32, so "data" and "precision" change together at 50M and 87M; Section 11.3 supports parity only for the 12M recipe at 1×. The NCM and AR see the same windows but not the same compute: the NCM stage costs more per window, and the model grows with the width. The codec was trained at the 12M scale and not adapted; the corpus is synthetic; samples are 128-token unconditional openings; and there is one external scorer and no human rating.

11.5 Natural text: a FineWeb-Edu pipeline check

We also ran the pipeline once on web text (E26): a new codec and NCM generators of 101.6M and 255.1M parameters, and a 100M-class AR baseline, trained on 800,000 windows of 128 tokens from one shard of FineWeb-Edu (Penedo et al. 2024), for one seed and early checkpoints of long schedules (the learning rate had barely decayed; about 5% of a compute-optimal token budget), and scored with a smaller protocol than Section 10′s. Every model is far from real text, no output is coherent prose, and the NCM scored about three times the AR's perplexity; with one seed, a batch-size confound between the two NCM sizes and a data preparation that is not in the repository, we draw no conclusion from that number and offer the run only as evidence that the pipeline works on natural text. The settings and values are in `docs/EXPERIMENTS.md`.

11.6 What these follow-ups establish and what remains open

They establish that, on synthetic stories with a frozen codec, the NCM/AR perplexity ratio falls as the generator grows and as it sees more data (3.33 to 2.13 over the grid), with the first size step and the extra data carrying most of the gain; that two further training-side changes and every sampling policy tried leave the ratio at 3.0 or above; and that bf16 mixed precision reproduces fp32 quality for the 12M recipe while being about 1.5 times faster per step in our measurement.

They do not establish a scaling law (three sizes, two data amounts, one training seed per cell except one); whether the trend continues, flattens or reverses at larger sizes; behaviour at matched total compute; parity of bf16 at 50M, 87M or four times the data; the role of the frozen codec; quality on natural text (the FineWeb run is a pipeline check); any conditional task; or human-rated quality.

12 Discussion

What carries over to a laptop. Measured Apple Silicon runs show lower small-batch NCM latency than an AR model of similar generator size on the tested synthetic task. The advantage shrinks with batch size and code count; historical 12-level Sudoku generation is slower than AR from batch 64 on. These observations do not establish natural-text speed or isolate dispatch, memory traffic and kernel costs. The completed E19/E20 training runs use CUDA, not the Mac. On TinyStories, the eager MLX NCM has lower recorded latency than the cached MLX AR baseline at batch sizes 1, 16 and 64 in one seed-0 run, while generating text that scores far worse (Section 10.8). That is a quality/latency tradeoff on a contended desktop, not a matched-quality speed result.

Partial success at the tested scale. A 6.44M generator learns substantial puzzle-conditioned structure: both funded-recipe seeds solve more than 97% of easy boards and 79% of medium boards. Both exceed E15′s reported rates, with twice its generator exposure and a changed overall recipe. This is sufficient evidence to continue studying the implementation. It is not an estimate of minimum required capacity, a scaling law or a demonstration of human-style logical reasoning. The additional codec prevents treating generator size as whole-system size.

Why hard Sudoku remains unresolved. The hard benchmark provides 30 clues, leaving 51 blanks, versus 40 clues and 41 blanks on easy boards. Every constraint and input clue must be satisfied simultaneously. Errors can be correlated, and a small change in average cell accuracy can accompany a larger change in exact-board accuracy. Neither clue count nor these aggregates reveals which logical dependencies the model fails to represent. The reference comparison also changes capacity, exposure, representation, corpus and difficulty-specific training. Those are candidate explanations, not experimentally established causes. Earlier width-512 trials do not justify promising that widening alone closes the gap.

How to challenge the interpretation. Before further long runs, retain checkpoint-bound input identities, per-board outputs and per-level diagnostics. Use a fresh untouched test set after fixing the selection protocol, since repeated examination of these benchmarks can guide recipe choices. Additional seeds characterize variation; separate random streams and controlled interventions are needed to identify whether initialization, order, dropout or corruption matters. True-code interventions on a diagnostic subset can test error propagation. Matched-data capacity and exposure ablations can test scale without conflating both. Checkpoint selection must use validation data rather than the external benchmark. These are proposed tests, not completed findings.

What the text experiments show. Two things were previously proposed and are now observed within bounds. The released checkpoints can be converted and run independently, with agreement to upstream PyTorch in the tested computations (Section 9); this validates inference compatibility and nothing about training. And an independently trained small NCM on synthetic TinyStories prose reconstructs text accurately but generates it poorly relative to a token-by-token baseline of the same block stack and window exposure (Section 10). The corruption ablation is the informative part of that result: lower clean teacher-forced loss accompanied worse free-running text, in both generator seeds. This echoes the Sudoku observation that validation loss can move against generation quality (Section 8), and it argues for reporting reconstruction, teacher-forced loss and free-running quality as separate columns. Two generator seeds that share one codec cannot say how large the variation across full training runs is, and no experiment here separates corruption strength from the other differences between schedules.

What the follow-ups show. On this synthetic task, with this codec, the gap to the autoregressive baseline responds to capacity and data and not to the training-side and sampling-side adjustments we tried (Section 11). Cooling the fine levels buys a few percent of the gap. Stronger corruption hurts, a constant corruption of the same expected amount matches the taper, and equal level weighting changes nothing resolvable. The bigger generator and four times the data lower the ratio from 3.3 to about 2.1. The autoregressive baseline improves as well, so the NCM's absolute gain (48.964 to 24.404) overstates its gain relative to the baseline. The data suggest a mundane reading, that the NCM at these sizes is capacity- and exposure-limited, and they do not exclude other readings: the codec is frozen and was fitted at the 12M scale, the exposure is small, the scorer favours fluent text, and a single seed per cell leaves the later size steps unresolved. That the ratio stays above two in every cell is better supported than any single step. How the ratio behaves at 100M parameters and billions of tokens is what this study cannot say. We read the bf16 result the same way: it removes one reason not to scale, for the recipe checked, and nothing more.

How to challenge the text interpretation. A small blinded human comparison of the retained outputs, with recorded instructions and sample identities, would test what the external scorer cannot. Novelty checks beyond exact openings would address memorisation. Independent codec seeds would test whether the result depends on the single frozen codec. A per-level comparison of teacher-forced and self-generated error would probe the proposed explanation, and a sampling-sensitivity study would separate poor sampling calibration from poor learned generation. A comparison at matched total compute, a conditional task and naturally occurring text would be needed before any broader claim. These are proposed tests, not completed findings. Nothing in Sudoku or TinyStories scores implies instruction following, summarisation, translation or application value.

13 Limitations

Most generator recipes use at most 6.4M parameters; width-512 ablations use about 25M. E19-funded and E20 provide only two training seeds of one recipe, not a reliable population variance estimate. The best hard score remains 45.80 points below Original NCM; the second seed remains 52.85 points below. The TinyStories study uses one frozen codec shared by both generator seeds, so it measures generator variability only; total training compute and model size are not matched between the MLX NCM and the MLX AR baseline; the corpus is synthetic GPT-4 prose; generation is unconditional and no conditional task is tested; scoring relies on a single external model without human ratings, and sampling-policy sensitivity, full-training-seed variability and memorisation beyond exact openings are untested. The aggregation sensitivity audit and the latency measurements cover generator seed 0 only. The Released TinyStories checkpoint is an upstream-trained reference at a far larger budget, not a reproduction target. For Sudoku, no AR comparison at the current 16.384M-presentation budget is available. No natural-text validation or matched total-cost study is available. Full-depth codec validation reconstruction does not prove useful coarse representations under generated errors. Historical aggregate-only scoring, missing remote manifests, repeated benchmark-guided development and unknown benchmark contention limit independent audit and generalization. The original training implementation is not released; unstated recipe details remain uncertain. The scaling study (Section 11) has one training seed per cell except one, and the differences between the 50M and 87M generators are within seed and hardware variation; its rows mix fp32 (1× data) and bf16 (4× data) with

parity shown only at 12M and 1× data, run on different GPUs, and do not match total compute between the NCM and the AR baseline or between sizes; the codec is frozen at the 12M scale; the sampling-policy driver and the FineWeb data preparation are private, so those results cannot be reproduced from the repository; the FineWeb pilot compares early checkpoints of long schedules with a smaller scoring sample, and its stronger-corruption variant has no result; and the matched-constant control has one seed. The XSum port is checked against upstream on CPU with greedy decoding only, and its guided row uses a different guidance weight from the paper's. Existing measurements establish neither production reliability nor a benefit in a user application.

14 Related work

Parallel and coarse-to-fine decoding for text includes discrete diffusion (Sahoo et al. 2024) and the methods compared in (Kumbong et al. 2026). Residual quantisation is standard in audio codecs (Zeghidour et al. 2021) and image generation (Lee et al. 2022; Tian et al. 2024). Scheduled sampling (Bengio et al. 2015) addresses the train–test mismatch that conditioning corruption targets. Bakhaow/ncm-forge adds PyTorch training loops around upstream modules. The repository's dated related-work comparison records other implementations; we make no priority claim for this MLX implementation. The released TinyStories and Sudoku checkpoints (Section 9) are the work of Kumbong et al. (Kumbong et al. 2026); our port only runs them. TinyStories text comes from the dataset of Eldan and Li (Eldan and Li 2023), used here at the pinned revision of Section 10 and scored with GPT-2 large (Radford et al. 2019).

15 Roadmap

The project's long-term aim is a text model of about one billion parameters whose generations are not worse than those of an autoregressive baseline trained with the same compute. Nothing in this paper shows that NCM can reach it. At the small scale tested in Section 10, the best MLX NCM run has about three times the GPT-2 large generative perplexity of the MLX AR baseline (45.568 against 14.311). In the larger runs of Section 11 the ratio falls to about 2.1 (24.404 against 11.484 at 87M parameters and four times the data) and stays above two. The steps below are plans, ordered by dependency, with no delivery dates; each later step is conditional on the evidence from the earlier ones.

Completed. TinyWorld, a synthetic grammar with codec, generator and AR baseline (Section 7). Sudoku, from E6 to the two-seed E19 and E20 recipe (Sections 7 and 8). The inference-only port of the released checkpoints, with parity checks, including XSum with ROUGE (Section 9). The TinyStories corruption ablation, E21 to E24, with two generator seeds (Section 10). The bf16 parity check, the size-by-data grid on TinyStories, the corruption and sampling follow-ups and a FineWeb-Edu pipeline check at about 100M and 250M parameters (Section 11).

Planned, in order.

1. Understand the remaining gap, which is above two at the largest size tried. Candidates, none tested yet: the recipe details that the released configuration and the paper fix (4,096 codes per level, a wider codec, a lower learning rate), the Muon optimiser and corruption by neighbouring codes, all three available as options in the software released with this version; independent codec seeds and a codec trained with the generator's size; more seeds per cell; a human comparison of outputs; and a design that changes the algorithm (iterated refinement). Further changes to the uniform corruption schedule, the loss weighting and the sampler had no effect in Section 11 and are not repeated.
2. Prompt-conditioned (prefix) training of our own models, and the released WMT14 generator. Red-fuchs described an MLX port of the official XSum and WMT14 codecs and a prefix-conditioned training loop in issue 3 of the code repository (Red-fuchs/ncm-modular); the codecs and the training option exist in software 0.3.0 and the released XSum generator is ported (Section 9), but no prefix-conditioned model of ours has been trained.
3. Mixed-precision training throughput. Done for the 12M recipe (Section 11.3); parity at larger sizes and more data is untested.
4. A natural-text corpus of several billion tokens, with its licence and splits recorded.
5. A model of about 100M parameters trained on about 2B tokens, as the go/no-go comparison against an AR baseline. Section 11 shows the pilot-scale data and compute are too small to answer it.

6. About 300M parameters, then about 1B, only if NCM is close to the AR baseline at matched compute.

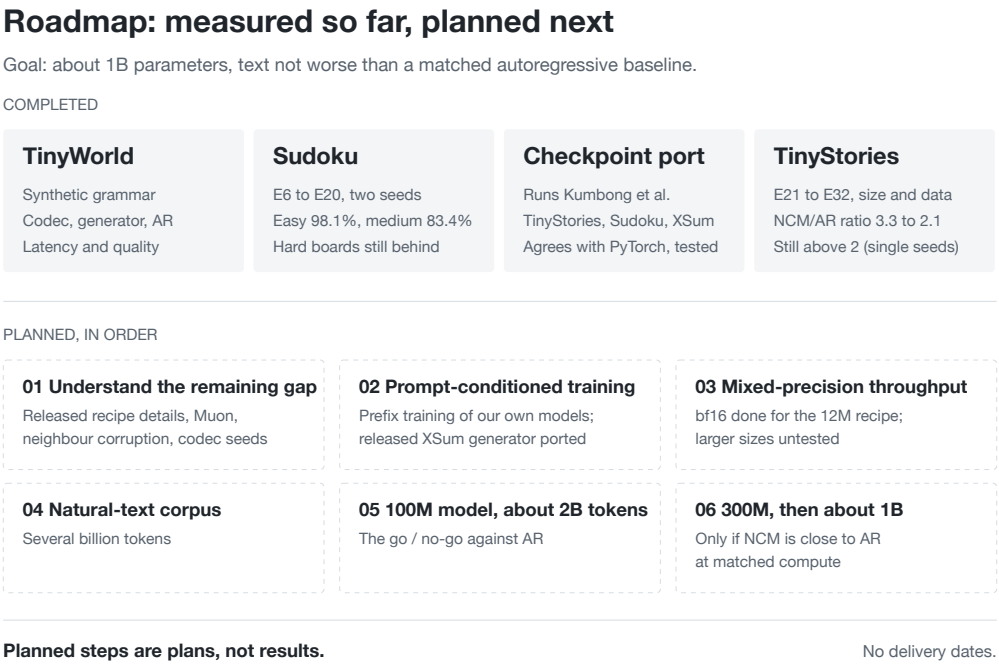


Figure 9: Project roadmap: completed milestones and planned steps. Planned steps are plans, not results.

16 Conclusion

This independent MLX implementation preserves exact CPU training/per-level inference equivalence and demonstrates coarse-to-fine generation on a finite synthetic language and Sudoku. The funded small-generator recipe reports 97.25–98.10% easy, 79.60–83.40% medium and 33.25–40.30% hard solving across two seeds at equal exposure. We regard this as partial implementation and small-model success, with substantial failures on hard boards and unresolved provenance and reproducibility limits. It does not reproduce Original NCM's scores or establish matched-cost superiority. The observed seed difference warrants better diagnostics rather than attributing the deficit to a single component. An inference-only port runs the released checkpoints of Kumbong et al. and agrees with upstream PyTorch in the tested computations, up to stated precision effects. Small TinyStories models trained here reconstruct held-out windows accurately but generate text that an external scorer rates far worse than a similarly built autoregressive baseline (generative perplexity 45.568–97.776 against 14.311–14.676). Across two generator seeds on one frozen codec, reducing conditioning corruption lowered clean teacher-forced code loss and worsened free-running text. These text results come from synthetic prose, unconditional generation and unmatched total compute, and they establish no conditional capability. Follow-up runs with the same frozen codec show that the quality gap falls as the generator grows and sees more data: the ratio of NCM to AR generative perplexity goes from 3.33 (11.8M parameters) to 2.38 (87.4M) at the original data and to 2.13 with four times the data, while corruption and loss-weighting variants and sampling policies do not lower it materially (stronger corruption raises it), and bf16 training matches fp32 at 11.8M. These runs are single-seed except one, unmatched in compute and mixed in precision, and the step from 49.6M to 87.4M is smaller than the seed-to-seed difference measured at 11.8M, so they describe a trend, not a scaling law. The XSum generator released by Kumbong et al. also runs in the port and reproduces its published unguided ROUGE scores to within 0.1 and, at guidance weight 0.5, the guided ones to within 0.2. The unresolved gaps are the hard-Sudoku deficit, the autoregressive quality gap on text (still above two at the largest size), the single-codec dependence and the absence of human ratings and of well-trained models on naturally occurring text.

Code and reproducibility

Code, configurations and this paper: github.com/janmus1c/ncm-mlx. Every configuration is in `configs/`; measured runs are logged in `docs/EXPERIMENTS.md`. The benchmark guide explains budget matching, parameter accounting, synchronization and uncertainty. The TinyStories training and evaluation source that produced Section 10 is identified by per-file SHA-256 digests in `docs/research/e21-e23-evidence.json`, together with configuration, data, tokenizer and scorer identities, checkpoint hashes and per-seed scores. The development commit named there is not in the public history; 41 of its 44 digested training files are byte-identical in the public repository, and the other three, in the released-checkpoint port, also contain later prefix-cache, compilation and benchmark changes. The 44 digested training files, exactly as they were run, are attached to the paper release as the archive `ncm-mlx-text-training-source-d323f4f.tar.gz` (SHA-256 `7c5f9d6e08f70eda075a4d35556bd46a3ef4cc622756f36545915040879a0c64`) (download); the digest of each file inside it matches the evidence extract. The aligned evaluation and benchmark driver is not part of the repository or the archive. A clean checkout does not yet reproduce those runs end to end, and the released package `ncm-mlx` 0.2.0 did not include the text-training commands (they ship from 0.3.0); the commands below cover the Sudoku path. The Section 11 configurations are `configs/tinystories-ladder-*.json`, `tinystories-e21-bf16.json`, `tinystories-matched-constant.json`, `tinystories-stronger-corruption*.json` and `tinystories-equal-level-loss*.json` (output directories under `artifacts/`), and their evaluation numbers are in `docs/research/tinystories-scaling.json`. These runs used the repository's text-training workflow with its optional bf16 mixed precision (`train.precision`). The stronger-corruption and equal-level-loss runs additionally used gradient accumulation of 2×32 on a 12 GB card. Accumulation is an option of the unreleased source that ran them; the public code and configurations have no such option and train the same recipe as one batch of 64, which needs more memory and is not bitwise the same computation. The sampling-policy study (Section 11.2) is run by `scripts/sampling_study.py` with the 98-run list `scripts/sampling_study_runs.json`. It is the private script that produced the tables, cleaned up for release, and it has not been re-run on a clean checkout. The FineWeb-Edu data preparation is private and not in the repository, so Section 11.5 cannot be reproduced from it. The audited reports, sample outputs and scorer records for Section 11 are attached to the paper release as the archive `ncm-mlx-tinystories-evidence-e25-e32.tar.gz` (SHA-256 `cbe6ec001b53a6f09a2b2b8fed4fce711852379cb42f6c277129b851bdfe430d`, download); the numbers in `docs/research/tinystories-scaling.json` were read from its `report.json` files. Scorer records exist for E25 and E28 only; the other runs keep their per-seed perplexities in the reports. The archive holds no model weights. The XSum port runs as `python -m ncm_mlx official summarize --task xsum`, and `scripts/official_xsum_parity.py` reproduces the parity check against upstream PyTorch. Audited reports, sample outputs and per-output scorer records for all six runs are attached to the paper release as `ncm-mlx-tinystories-evidence-e21-e24.tar.gz` (SHA-256 `4a744f5306ebef3433abbab9647372d6ad00949f616c7befe9476fbccb0d20b8`, download).

```
git clone https://github.com/janmus1c/ncm-mlx && cd ncm-mlx
mise run setup                                  # Python 3.12, MLX, tests
scripts/run_experiment.sh configs/sudoku-e6.json   # train, evaluate, benchmark
.venv/bin/python paper/make_figures.py --plot      # rebuild figures from committed data
paper/build.sh                                  # rebuild this paper
```

AI assistance

OpenAI Codex assisted with implementation, experiments, analysis, figures, documentation and manuscript drafting and revision. It is not an author. This assistance does not constitute independent verification of the results or human review of the manuscript.

References

Bengio, Samy, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. 2015. 'Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks'. *Advances in Neural Information Processing Systems*.

Eldan, Ronen, and Yuanzhi Li. 2023. *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* https://arxiv.org/abs/2305.07759.

Hannun, Awni, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. 2023. *MLX: Efficient and Flexible Machine Learning on Apple Silicon*. Https://github.com/ml-explore/mlx.

Kumbong, Hermann, Ishan S. Khare, Iddah Mlauzi, et al. 2026. 'Next-Concept Modeling: Fast, Parallel Coarse-to-Fine Text Generation'. *arXiv Preprint arXiv:2610.14162*.

Lee, Doyup, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. 'Autoregressive Image Generation Using Residual Quantization'. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Lin, Chin-Yew. 2004. 'ROUGE: A Package for Automatic Evaluation of Summaries'. *Text Summarization Branches out*, 74–81.

Narayan, Shashi, Shay B. Cohen, and Mirella Lapata. 2018. 'Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization'. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.

Oord, Aaron van den, Oriol Vinyals, and Koray Kavukcuoglu. 2017. 'Neural Discrete Representation Learning'. *Advances in Neural Information Processing Systems*.

Penedo, Guilherme, Hynek Kydlíček, Loubna Ben Allal, et al. 2024. 'The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale'. *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models Are Unsupervised Multitask Learners*. OpenAI.

Rippel, Oren, Michael Gelbart, and Ryan Adams. 2014. 'Learning Ordered Representations with Nested Dropout'. *International Conference on Machine Learning*.

Sahoo, Subham Sekhar, Marianne Arriola, Yair Schiff, et al. 2024. 'Simple and Effective Masked Diffusion Language Models'. *Advances in Neural Information Processing Systems*.

Su, Jianlin, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. 'RoFormer: Enhanced Transformer with Rotary Position Embedding'. *arXiv Preprint arXiv:2104.09864*.

Tian, Keyu, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. 2024. 'Visual Autoregressive Modeling: Scalable Image Generation via next-Scale Prediction'. *Advances in Neural Information Processing Systems*.

Zeghidour, Neil, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. 'SoundStream: An End-to-End Neural Audio Codec'. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*.

Appendix A: Hyperparameters

Table 16: Main settings per experiment. All runs: AdamW, weight decay 0.01, cosine schedule with warmup, batch 128, commitment $\beta = 0.25$, fine-level dropout 0.5.

Run	Levels	Codes	Codec	Codec steps (lr)	Generator	Gen. steps (lr)	Other
E1	1,2,4,8,16,32	128	128 wide, 2+2 blocks	3,000 (1e-3)	6×256	3,000 (6e-4)	EMA 0.95; best step 1,500
E4	1,3,9,27,54,81	32	2 encoder blocks	3,000 (1e-3)	8×256	5,000 (6e-4)	prefix 81
E5	1,2,4,6,9,13,18, 26,37,50,67,81	16	lookup encoder, 1 decoder block, 256 wide	3,000 (1e-3)	8×256	4,000 (6e-4)	EMA 0.99

Run	Levels	Codes	Codec	Codec steps (lr)	Generator	Gen. steps (lr)	Other
E6	as E5	16	E5 + learned level 0, GroupNorm(8), dropout 0.3	3,000 (2e-4)	8 × 256	4,000 (6e-4)	
E8	as E5	16	E6 codec	—	8 × 256	4,000 (6e-4)	corruption 0.30 → 0.05, compiled
E9	as E5	16	E6 codec	—	8 × 256	4,000 (6e-4)	as E8, model-sampled

Later Sudoku runs reuse the frozen E6 codec and its 12-level, 16-code hierarchy. All use an 81-cell prefix, eight attention heads, configured generator seed 0 except E20 (seed 1), AdamW with weight decay 0.01, a cosine learning-rate schedule and compiled generator training. Generator dropout is 0.1 at width 256 and 0.2 at width 512; warmup is 200 steps at width 256 except E19/E20, which use 1,000; width 512 also uses 1,000. The codec is additional to the generator sizes below.

Table 17: Later Sudoku generator settings. Steps are completed steps for the reported results; corruption gives the first and last replacement probabilities.

Run	Generator	Steps	Batch	Learning rate	Corruption
E10	8 × 512	24,000	64	3e-4	0.30 → 0.05
E11	8 × 256	32,000	64	6e-4	0.30 → 0.05
E12	8 × 512	50,776	64	3e-4	0.50 → 0.10
E13	8 × 256	32,000	64	6e-4	0.50 → 0.10
E14	8 × 256	128,000	64	6e-4	0.30 → 0.05
E15	8 × 256	128,000	64	6e-4	0.50 → 0.10
E16	8 × 256	32,000	64	6e-4	0.50 → 0.10
E17	8 × 256	32,000	64	6e-4	0.50 → 0.10
E18	8 × 256	128,000	64	6e-4	0.50 → 0.10
E19 pilot	8 × 256	23,895	128	6e-4	0.50 → 0.10
E19 funded	8 × 256	128,000	128	6e-4	0.50 → 0.10
E20	8 × 256	128,000	128	6e-4	0.50 → 0.10

The base data are 200,000 training puzzles and 1,000 validation puzzles, with split seed 0 and 28–45 givens. E15 adds one million greedy-unique puzzles for each seed 201–204; E16 adds one million for each seed 205–206. E17 adds the unique subset of one million random-removal candidates for each seed 101–104; its final 2,000 steps are a weights-only continuation with reset optimizer and random state (Section 6.5). E12′s recorded run completed 50,776 steps; the repository's current E12 configuration specifies 200,000, so running that configuration unchanged does not reproduce the reported exposure. E18 adds no unique-puzzle corpus. Both E19 runs and E20 add one million greedy-unique puzzles per seed 201–208; their separate schedules use full-state snapshots every 1,000 updates. The original million-step E19 configuration remains unstarted and is not either reported result. Full settings and checkpoint prerequisites are in the corresponding `configs/sudoku-e*.json` files and `docs/EXPERIMENTS.md`.

TinyStories settings (Section 10). Codec: 12 levels of lengths 1, 8, 16, 24, 32, 40, 52, 64, 76, 92, 108, 128 with 2,048 codes each, width 256, 4 heads, 14,000 updates at batch 256, learning rate 5e-4 with cosine decay to 5e-5, warmup 1,000, weight decay 0.01, dropout 0.3, fine-level dropout 0.5, commitment 0.25, EMA decay 0.95. Generator: 6 × 384, 6 heads, dropout 0.1, 15,000 updates

at batch 64, warmup 500, weight decay 0.1, betas (0.9, 0.95), gradient clipping 1, loss $\alpha = 0$. AR baseline: same stack, 7,500 updates at batch 128. Corruption endpoints: 0.75 → 0.25 (E21, E24), 0.30 → 0.05 (E22) and 0 → 0 (E23); E24 and the seed-1 repeats of E22 and E23 differ only in configured seed 1 and reuse the seed-0 codec.

Section 11 settings. All runs: the TinyStories codec and settings above, with the frozen E21 codec reused. Generators: 6 × 384 × 6 heads (11,832,587 parameters), 12 × 576 × 9 heads (49,644,683) and 12 × 768 × 12 heads (87,426,059); the AR baseline of each size uses the same blocks (13,794,048, 52,586,880 and 91,348,992 parameters). Data multiple 1×: generator 15,000 updates at batch 64, AR 7,500 at batch 128, fp32. Data multiple 4×: generator 60,000 updates at batch 64 (3,840,000 windows), AR 30,000 at batch 128, `train.precision = "bfloat16"`. The bf16 parity run is the E21 recipe with `train.precision = "bfloat16"`. Corruption variants: matched constant 0.3878 (E25); 0.90 → 0.50 (E28); equal level loss $\alpha = 1$ with the E21 corruption (E28). The FineWeb pipeline check (E26) uses 14 × 768 × 12 heads and 20 × 1024 × 16 heads generators, batch 16 and 8, peak learning rate 3e-4, corruption 0.75 → 0.25, on 800,000 training windows of 128 tokens.

Appendix B: Environment

Apple M5 Pro, 64 GB unified memory, macOS 26.5; MLX 0.32.3, NumPy, Python 3.12.14 managed with mise and uv. Figures: matplotlib. Paper: Pandoc and Typst. E18: RTX 3080, Python 3.12.15; both E19 runs and E20: NVIDIA A40, Python 3.12.3, MLX 0.32.3 CUDA. Preflight and contention qualifications accompany each run in the experiment log. TinyStories experiments and parity checks: Apple M5 Pro, 64 GB, macOS 26.5, Python 3.12.14, MLX 0.32.3, `mlx-lm` 0.32.2, NumPy 2.5.4, PyTorch 2.15.0 for upstream parity only; Metal float32 with reduced-precision matrix products. Section 11 runs: the Apple M5 Pro (the matched-constant control and the earlier sampling study, MLX 0.32.3), an RTX 3080 (12 GB; the bf16 parity, the 12M four-times-data runs, the corruption and loss-weighting variants and the main sampling sweeps), a rented RTX A6000 (12M and 50M at the original data), a rented A40 (50M at four times the data), a rented RTX PRO 6000 Blackwell (87M at both data amounts) and a rented RTX 5090 (the FineWeb-Edu pilot), through MLX 0.32.3′s CUDA backend with Python 3.12.3 (3.12.15 on the RTX 3080); the XSum port was measured with MLX 0.32.4 on the Mac, and its parity check used PyTorch 2.15.0 and transformers 5.22.0 on CPU. The `rouge-score` package is 0.1.2.

Appendix C: Version history

Version	Date	Changes
0.1	2026-10-23	First draft: method, implementation, TinyWorld and Sudoku results E0–E6.
0.2	2026-10-23	Plain-language introduction, glossary, formulas explained with a running example, experiment journey (E0–E9), hyperparameter appendix, NeurIPS-style layout, HTML, EPUB, LaTeX, JATS and DOCX editions.
0.3	2026-10-23	E8 results: conditioning corruption gives the first solved boards.
0.4	2026-10-23	E11: four times the training data gives 55% of easy boards; first run on NVIDIA.
0.5	2026-10-24	E13 (stronger corruption) and E14 (16× data): 87% of easy boards.

Version	Date	Changes
0.6	2026-10-24	E15 combines data, stronger corruption and unique puzzles: 96% of easy boards. Layout checks on every build.
0.7	2026-10-24	Full-state pause/resume and recovery checks; E17′s weights-only continuation solves 71.25% of easy boards, with the optimizer-reset qualification.
0.8	2026-10-24	Release audit: codec/system parameter accounting, budget and sampling qualifications, prior-art scope and simplified visual identity.
0.9	2026-10-24	Submission preparation: public-data figure rebuild command, qualified memory-bandwidth discussion, later Sudoku settings and AI assistance disclosure.
0.10	2026-10-25	Completed E18/E19 results, checkpoint provenance, A40 timings, uncertainty and explicit budget differences.
0.11	2026-10-27	Completed E20 seed replication; two-seed assessment, hard-board failure accounting, explicit provenance limits and a gated text-research direction.
0.12	2026-11-02	Roadmap; inference parity for the released checkpoints; TinyStories training contract; corruption ablation (E21–E24) with two generator seeds on one frozen codec, AR, released and held-out controls, aggregation sensitivity and qualified latency; text future-work statements replaced by the observed bounded result. TinyWorld and Sudoku results unchanged.
0.13	2026-11-04	Scaling study in the TinyStories setup (generator size × data, E30–E32), bf16 quality parity (E29), negative follow-ups (matched-constant corruption, stronger corruption, equal level loss, sampling policies; E25, E27, E28), FineWeb-Edu pipeline check (E26), released XSum generator with parity and ROUGE (Section 9.4); abstract, discussion, limitations, roadmap and conclusion updated. v0.12 content, TinyWorld and Sudoku results unchanged.